TEXT MINING WITH MACHINE LEARNING

Principles and Techniques



Jan Žižka, František Dařena, Arnošt Svoboda

CRC Press
Taylor & Francis Group
A SCIENCE PUBLISHERS BOOK

Text Mining with Machine Learning Principles and Techniques

Jan Žižka
Machine Learning Consultant, Brno, Czech Republic

František Dařena
Department of Informatics Mendel University, Brno, Czech Republic

Arnošt Svoboda
Department of Applied Mathematics & Computer Science Masaryk University, Brno, Czech Republic

CRC Press
Taylor & Francis Group
Boca Raton London New York

CRC Press is an imprint of the
Taylor & Francis Group, an **informa** business

A SCIENCE PUBLISHERS BOOK

CRC Press
Taylor & Francis Group
6000 Broken Sound Parkway NW, Suite 300
Boca Raton, FL 33487-2742

© 2020 by Taylor & Francis Group, LLC
CRC Press is an imprint of Taylor & Francis Group, an Informa business

No claim to original U.S. Government works

Version Date: 20190826

International Standard Book Number-13: 978-1-138-60182-6 (Hardback)

This book contains information obtained from authentic and highly regarded sources. Reasonable efforts have been made to publish reliable data and information, but the author and publisher cannot assume responsibility for the validity of all materials or the consequences of their use. The authors and publishers have attempted to trace the copyright holders of all material reproduced in this publication and apologize to copyright holders if permission to publish in this form has not been obtained. If any copyright material has not been acknowledged please write and let us know so we may rectify in any future reprint.

Except as permitted under U.S. Copyright Law, no part of this book may be reprinted, reproduced, transmitted, or utilized in any form by any electronic, mechanical, or other means, now known or hereafter invented, including photocopying, microfilming, and recording, or in any information storage or retrieval system, without written permission from the publishers.

For permission to photocopy or use material electronically from this work, please access www.copyright.com(`http://www.copyright.com/`) or contact the Copyright Clearance Center, Inc. (CCC), 222 Rosewood Drive, Danvers, MA 01923, 978-750-8400. CCC is a not-for-profit organization that provides licenses and registration for a variety of users. For organizations that have been granted a photocopy license by the CCC, a separate system of payment has been arranged.

Trademark notice: Product or corporate names may be trademarks or registered trademarks, and are used only for identification and explanation without intent to infringe

Library of Congress Cataloging-in-Publication Data

Names: Žižka, Jan, author. | Dařena, František, 1979- author. | Svoboda, Arnošt, 1949- author.
Title: Text mining with machine learning : principles and techniques / Jan Žižka, František Dařena, Arnošt Svoboda.
Description: First. | Boca Raton : CRC Press, 2019. | Includes bibliographical references and index. | Summary: "This book provides a perspective on the application of machine learning-based methods in knowledge discovery from natural languages texts. By analysing various data sets, conclusions, which are not normally evident, emerge and can be used for various purposes and applications. The book provides explanations of principles of time-proven machine learning algorithms applied in text mining together with step-by-step demonstrations of how to reveal the semantic contents in real-world datasets using the popular R-language with its implemented machine learning algorithms. The book is not only aimed at IT specialists, but is meant for a wider audience that needs to process big sets of text documents and has basic knowledge of the subject, e.g. e-mail service providers, online shoppers, librarians, etc." – Provided by publisher.
Identifiers: LCCN 2019035868 | ISBN 9781138601826 (hardback)
Subjects: LCSH: Machine learning. | Computational linguistics. | Semantics–Data processing.
Classification: LCC Q325.5 .Z59 2019 | DDC 006.3/12–dc23
LC record available at https://lccn.loc.gov/2019035868

Visit the Taylor & Francis Web site at
http://www.taylorandfrancis.com
and the CRC Press Web site at
http://www.crcpress.com

To our families

Contents

Preface **xi**

Authors' Biographies **xiii**

1 Introduction to Text Mining with Machine Learning **1**
1.1 Introduction . . . 1
1.2 Relation of Text Mining to Data Mining . . . 2
1.3 The Text Mining Process . . . 6
1.4 Machine Learning for Text Mining . . . 7
1.4.1 Inductive Machine Learning . . . 9
1.5 Three Fundamental Learning Directions . . . 10
1.5.1 Supervised Machine Learning . . . 10
1.5.2 Unsupervised Machine Learning . . . 11
1.5.3 Semi-supervised Machine Learning . . . 11
1.6 Big Data . . . 12
1.7 About This Book . . . 13

2 Introduction to R **14**
2.1 Installing R . . . 15
2.2 Running R . . . 16
2.3 RStudio . . . 18
2.3.1 Projects . . . 19
2.3.2 Getting Help . . . 20
2.4 Writng and Executing Commands . . . 21
2.5 Variables and Data types . . . 23
2.6 Objects in R . . . 24
2.6.1 Assignment . . . 27
2.6.2 Logical values . . . 28
2.6.3 Numbers . . . 29
2.6.4 Character Strings . . . 30
2.6.5 Special values . . . 32

2.7 Functions . . . 34
2.8 Operators . . . 38
2.9 Vectors . . . 39
2.9.1 Creating Vectors . . . 39
2.9.2 Naming Vector Elements . . . 41
2.9.3 Operations with Vectors . . . 43
2.9.4 Acessing Vector Elements . . . 45
2.10 Matrices and Arrays . . . 47
2.11 Lists . . . 51
2.12 Factors . . . 53
2.13 Data frames . . . 56
2.14 Function Useful in Machine Learning . . . 60
2.15 Flow Control Structure . . . 67
2.15.1 Conditional Statement . . . 67
2.15.2 Loops . . . 70
2.16 Packages . . . 72
2.16.1 Installing Packages . . . 72
2.16.2 Loading Packages . . . 73
2.17 Graphics . . . 73

3 Structured Text Representation 82
3.1 Introduction . . . 82
3.2 The Bag-of-Worlds Model . . . 86
3.3 The Limitations of the Bag-of-Words Model . . . 88
3.4 Document Features . . . 92
3.5 Standardization . . . 94
3.6 Texts in Different Encodings . . . 99
3.7 Language Identification . . . 101
3.8 Tokenization . . . 102
3.9 Sentence Detection . . . 103
3.10 Filtering Stop Words, Common, and Rare Terms . . . 104
3.11 Removing Diacritics . . . 109
3.12 Normalization . . . 109
3.12.1 Case Folding . . . 110
3.12.2 Stemming and Lemmatization . . . 110
3.12.3 Spelling Correction . . . 113
3.13 Annotation . . . 115
3.13.1 Part of Speech Tagging . . . 115



Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

Preface

The aim of this book is to make it easier for those interested in text mining - one of the branches of modern information technology - to enter an interesting area focused on analyzing texts in natural languages. Machine learning has been chosen as an analytical tool because it is a promising organic part of computer science and artificial intelligence that supports successful exploration of big volumes of text data using advanced classification and clustering methods. Machine learning now has available a large number of software tools, both commercial and free, so there is no need to implement them and a researcher can focus directly on their use for given data and applications.

The authors of this book have chosen demonstrations of certain basic as well as popular implementations of machine learning algorithms in an advanced interpreted programming language R, which is freely available for a variety of operating systems and provides an appropriate communication interface over RStudio without the need for complex learning. In no case is this book primarily focused on advanced computer scientists, programmers, or mathematicians, although basic programming skills and higher mathematical knowledge (linear algebra, statistics, function analysis) are required to understand algorithms, use them and, of course, interpret their results. For interested readers, chapters have references to literature which can suitably complement the topics described. Text mining has already come to an advanced/a well-developed form, driven by the need to explore texts as specific data type, in particular large text data files on the Internet from social networks, product or service reviews, e-mails, blogs, and many forums. Perhaps every e-mail user today is aware of the facility to filter messages (for example, spam removal or topic grouping), which is just one of many possible applications of the text mining industry.

The book covers the introduction to text mining by machine learning, introduction to the R programming language, structured text representation, classification and easy introduction to the most common classification algorithms (Bayes classifier, nearest neighbors, decision trees, random forest, sup-

port vector machines, and deep learning), followed by clustering. The final two chapters deal with word embedding and feature selection problem areas. The authors of this book would like to express their hope that the book will find a number of interested readers and will be used to familiarize them with the attractive sector of artificial intelligence and machine learning applied to the difficult area of processing data produced by humans and not by machines.

Authors' Biographies

Jan Žižka is currently involved in consultations in the field of machine learning and data mining. For many years he worked as a system programmer, developer of advanced software systems, and researcher. For the last 25 years, he has devoted himself to artificial intelligence and machine learning, especially text mining. He has also worked at a number of universities in the Czech Republic and abroad, including research institutes. He is the author and co-author of approximately 100 international publications.

František Dařena works as an associate professor, the guarantor of the doctoral degree program System Engineering and Informatics, and head of the Text Mining and Natural Language Processing group at the Department of Informatics, Faculty of Business and Economics, Mendel University Brno. He is the author of several publications in international scientific journals, conference proceedings, and monographs; member of the editorial boards of international journals, and editor-in-chief of International Journal in Foundations of Computer Science & Technology. His research areas include text/data mining, intelligent data processing, and machine learning.

Arnošt Svoboda is an expert in programming and his area includes programming languages and systems such as R, Assembler, Matlab, PL/1, Cobol, Fortran, Pascal, and others. He started his career as a system programmer. For the last 20 years, Arnos˘t has worked as a teacher and researcher at Masaryk University in Brno (Czech Republic), Faculty of Economics and Administration, Department of Applied Mathematics and Computer Science. At present, he is interested in machine learning and data mining



Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

Chapter 1

Introduction to Text Mining with Machine Learning

1.1 Introduction

In the last few years, we have seen an explosion in the amount of data as the result of many activities. The data comes from various sources and is available in many formats. This has been enabled, primarily, by the massive advance on the Internet. Various devices like smart phones enable communication and the use of applications anytime and anywhere; many activities like shopping, interacting with government institutions, or providing support to customers, are moving to digital environments; many documents are being digitalized; people meet and interact on virtual platforms. The transformation of the Web into Web 2.0 [26], where the content is actively created by users, has thrown open to the masses, many possible avenues to express their ideas, recommendations, or attitudes.

It is obvious that texts written in a natural language are a natural way of human communication. Textual documents are thus strongly related to many human activities and they have become a source that is worth analyzing. The results of such an analysis can bring useful insights in practically all domains.

As the ability of people to analyze texts remains essentially the same, the availability of more data requires new computational methods in order to find something useful in large collections of documents. Thus, a discipline known as text mining has become very popular and attractive. Text mining can be defined as a knowledge-intensive process in which a user interacts with a collection of documents by using analytic tools in order to identify and explore

interesting patterns [87]. Applications can be found in marketing, competitive intelligence, banking, health care, manufacturing, security, natural sciences, and many other domains [254, 192].

Computers are able to analyze only the syntactic aspect of texts, which means that they are able to recognize how words are arranged in the documents. Because texts in a natural language are written using a grammar, some syntactic patterns in a text can be more or less easily identified. Semantics refers to the meaning of a word or group of words in a context. Without perfect understanding of a language it is not possible to completely understand the meaning. Fortunately, is is possible to solve many practical problems even without the full understanding of a text because syntax and semantics are often closely related. If two texts use the same words and syntactic structures, they are likely to be semantically similar and can, for example, belong to the same class of documents [196].

Documents can generally be analyzed in two different ways. The statistical or machine learning approach uses mathematical representation of texts. Linguistic methods, using natural language processing techniques, represent texts using language models where the meaning and different relationships are contained. Text mining uses both approaches to find knowledge, usually in a large number of texts [254, 271].

1.2 Relation of Text Mining to Data Mining

Text mining encompasses a wide variety of tasks that can bring information about different aspects of texts. The typical text mining tasks include [196, 279, 123]:

- categorization of documents - assigning a document to one or more predefined category (e.g., assigning a newspaper article to one or more categories, labeling e-mails as spam or ham);
- clustering - grouping documents according to their similarity, for example, in order to identify documents sharing a common topic;
- summarization - finding the most important parts in one or more documents and creating a text that is significantly shorter than the original;
- information retrieval - retrieving documents that match a query representing information needed from a large collection of documents;
- extracting the meaning of documents or their parts - identifying hidden topics, analyzing sentiment, opinion, or emotions;
- information extraction - extracting structured information like entities,

events, or relations from unstructured texts;
- association mining - finding associations between concepts or terms in texts;
- trend analysis - looking at how concepts contained in documents change in time;
- machine translation - converting a text written in one language to a text in another language.

Some of the text mining tasks are very similar to the tasks of data mining. Data mining is the automatic or semiautomatic process of finding implicit, previously unknown, and potentially useful knowledge in collections of electronically stored data. The knowledge has a form of structural patterns in data that can be also used to make predictions or provide answers in the future [280].

Data mining includes many different methods, tools, algorithms, or models. All of them require the data to be in a structured form. This means that the data can be represented in a tabular form as in a relational database. The data takes the form of a set of examples (or instances, data points, observations) described by specific values of their features (or attributes, variables, fields).

The features can be of several types [75, 174]:

- categorical (nominal) - the domain is a discrete set of values where ordering does not make sense;
- binary - a special type of categorical attribute with only two possible values;
- ordinal - the domain is a discrete set of values that can be ordered;
- numerical - the value of a feature is a number, either integer or continuous.

An example of a structured representation of data describing purchases in a retail store is in Table 1.1. Every purchase is characterized by a customer (described by age, education level, and gender), date of purchase, and the total price.

Such a format is, however, not typical for texts. Generally, a text is a string written in a natural language consisting of parts (words) with certain meaning that are combined according to some rules (syntax). The texts can also be of a different range. A unit of a text can be a sentence, a few sentences combined in a paragraph, or much longer texts, like web pages, e-mails, articles, or books. Sometimes, a text can be just a few words that are not a valid sentence, which is quite typical, for example, for short posts on social

networks.

Table 1.1: A structured representation of purchases in a retail store.

Age	Gender	Education	Date	Price
35	female	primary	$2019-02-10$	20.0
40	male	tertiary	$2019-02-14$	28.4
21	male	secondary	$2019-01-30$	15.1
63	female	secondary	$2019-03-01$	11.9

In order to be able to apply data mining methods to texts, they need to be converted to a structured representation. A classic structured representation of texts as vectors in a vector space model is known as bag-of-words and the process of inferring this representation is described in chapter 3. Another, more modern, representation that is based on embedding words to a continuous vectors space referred as to word embeddings is discussed in Chapter 13.

There are a few problems related to the bag-of-words representation, which are not that typical for data mining tasks in general [134, 65, 188, 170, 202]:

- Independently of the complexity of features, the input space for text mining problems is large. It is not uncommon that the dictionaries of various natural languages contain hundreds of thousands of words. When considering not only words as features but also, for example, combinations, mutual positions, or grammatical relationships between words, the complexity further increases. Of course, not all words of a language will appear in most document collections. On the other hand, even relatively small collections can contain tens of thousands of unique words (see 1.1).
- The data contains a lot of noise which is often given by the nature of the data. Text documents are usually created by people that make many errors, including mistyping, grammatical errors, and incorrect labeling (e.g., assigning one star instead of five stars to a positive review).
- The vectors representing documents are very sparse. If we compare the number of unique words in one document (tens of words for short messages) to the dictionary size (tens of thousands), the difference is enormous. Thus, only a small fraction of vector elements are non-zero values (see Figure 1.1).
- The distribution of the probability with which the words appear in doc-

uments is strongly skewed, following so called Zipf's law (see Section 3.3). This means that the majority of the content is represented with just a few words and that most of the words appear with very small frequencies.

- In many tasks, only a small fraction of the content is often relevant. For example, for a document classification task, only a few hundred words from the dictionary are needed in order to correctly assign documents to their classes with a relatively high accuracy. Many of the words are therefore redundant.

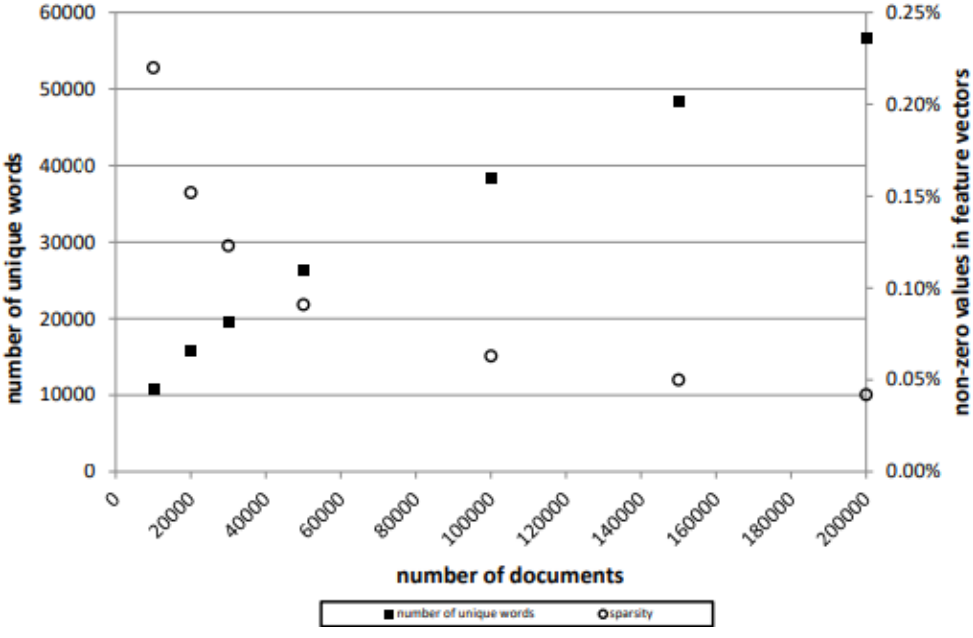


Figure 1.1: The graph represents a dependency between the number of documents and the number of unique words in a document collection. The documents represent customer reviews with the minimal, maximal, and average lengths equal to 1, 400, and 24 words.

- For many tasks, there often exists a scarcity of labeled data. This is because the labeling process, usually performed manually, is very demanding. A special situation is that there are instances of positive training data for a specific problem, but negative examples are missing.
- In many situations, the distribution of the data in terms of classes or topics is not balanced. This means, that some classes have many more

instances than the others which is generally a problem for learning. An example is the field of sentiment analysis, where the positive reviews in certain product categories are much more common than the negative ones.

1.3 The Text Mining Process

To find useful knowledge in a collection of text documents involves many different steps. To arrange them into a meaningful order, we might look at the general text mining process. It consists of the following steps [271, 133, 196]:

- Defining the problem: This step is actually independent of any actions which may subsequently be taken. Here, the problem domain needs to be understood and the questions to be answered, defined.
- Collecting the necessary data: The sources of texts containing the desired information need to be identified and the documents collected. The texts can come from within a company (internal database or archive) or from external sources, from the web, for example. In this case, web scrapers need to be frequently implemented to directly grab the content of the web pages. Alternatively, the API of some web based systems can be used to retrieve the data. After retrieval, texts are stored so they are ready for further analysis.
- Defining features: Features that well characterize the texts and are suitable for the given task need to be defined. The features are typically based on the content of the documents. A very simple approach, bag-of-words with binary attribute weighting, takes every word as a boolean feature. Its value indicates whether or not the word is in a document. Other methods might use more complicated weighting schemes or features that are derived from the words (modified words, combinations of words, etc.).
- Analyzing the data: This is the process of finding patterns in data. According to the type of task to be solved (e.g., classification), a specific model or algorithm is selected and its properties and parameters defined. Then, the model can be applied to the data so a solution to the solved problem can be found. To solve a specific problem, more models are usually available. The choice is not explicitly given in advance. The models have different characteristics that influence the data mining process and its result. The model can be (white box) or does not have to be (black

box) well interpretable by a human. Some models have higher computational complexity than the others. According to the utilization of the model, fast creation can be preferred over fast application or vice versa. The suitability of a model is often strongly dependent on the data. The same model can provide excellent results for one data set while it can completely fail for another. Thus, selecting a proper model, finding the right structure for it, and tuning the parameters often requires a lot of experimental effort.

- Interpreting the results: Here, some results are obtained from the analysis. We need to carefully look at them and relate them to the problem we wanted to solve. This phase might include verification and validation steps in order to increase the reliability of the results.

1.4 Machine Learning for Text Mining

Machine learning [199] as part of computer science and informatics is one of the most practically oriented areas of artificial intelligence [161]. It was inspired by the ability to learn - in other words, to gain new or additional knowledge, which is one of important characteristics of living organisms. Machine learning as a science focuses on looking for and developing algorithms, which can simulate or emulate mental abilities of living organisms from the learning point of view.

Learned knowledge may not completely address everything - for it is something of a general nature - but, it allows us to solve future problems that did not occur frequency, in the same form and/or environment in the past. If such a future problem is similar to something which has already occured, it is often not possible to solve it using the exact instructions from the past; however, a similar solution (or procedure) can hopefully be successful in most (or ideally, all) cases that are yet to occur.

Machine learning is one of the modern tools that includes anything which can be useful. Today, it consists of tens of various algorithms, and research constantly brings modified or completely new algorithms, sometimes replacing the older ones because the field of machine learning is strongly dominated by the practical needs of the real world that are growing in time. In addition to algorithms and relevant information technologies, machine learning inevitably uses mathematics, in particular the theory of probability, statistics, combinatorics, mathematical analysis, and a set of other essential disciplines.

To solve problems with help of computers, machine learning differs from the traditional method, which depends on applying carefully implemented

and debugged programs to input data. The traditional computer program consists of sets of instructions that process the input data and provide the output using a set of predefined parameters, which are provided externally. An example is a computer program that searches for a pre-set list of solutions to expected problems, something like a table containing results for combinations of input values or a set of mathematical equations. Such a program can be-in a simplified wayconsidered as a function $f(x) = y$, which for a given input x returns an output y, however, provided that $f(x)$ is reliably known. Without knowing such a function, it is necessary to estimate or approximate it.

If the needed conditions and rules are observed, the result is guaranteed because it is based on mathematically proved theorems and methods. Unfortunately, if there are-often even very small- changes in data or working environment, such a program is predisposed to a fundamental error or malfunction. For example, analyzed text data may begin to contain new terms that were not known when the program was created, or it becomes necessary to start analyzing such data in another language. In such a case, it is inevitable to rework, often completely, that program.

Machine learning can use an existing implemented algorithm, retrain it with other examples and apply such new or extended knowledge to solving a new problem. A certain defect might be that this knowledge is not supported enough by mathematical proofs - in machine learning, the support comes from empirical proofs and heuristics.

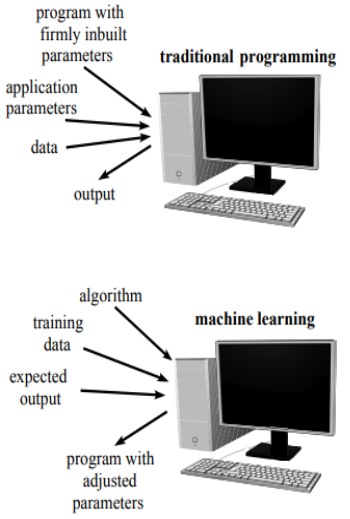


Figure 1.2: Traditional programming versus machine learning - illustration of the principles.

A simple illustration of the difference between traditional methods and machine learning is shown in Figure 1.2.

Determining the structure of models and their parameters is the goal of the training process, which uses (often iteratively) the presented training samples for this purpose. The success rate of the training phase must be measured by testing, which uses another set of samples which are not part of the training process.

1.4.1 Inductive Machine Learning

Current methods, oriented on solving practical problems for example [221, 208, 75], are particularly based on inductive approaches. The inductive learning is based on the generalization of (many) specific samples for a given problem area. The samples are comprised of collected data. For purpose of this book, that part of data, which is relevant for solving a given problem, is called information. Then, generalized information forms knowledge. The main task of machine learning is mining knowledge hidden in large collections of data.

Having a set of n samples of data items, $\{x_1, x_2, \ldots, x_n\}$, and knowing for each sample x_i its correct result y_i, the approximate function $\hat{f}(x) \approx f(x)$. The accuracy of such approximation depends naturally on the quality of samples how well they can cover (or represent) all possible data items. In

the real world, typically, not all possible data items are available for a given moment, or there might be so high (whether final or infinite) a number of such items that it is practically impossible to process them within an available time period. At the very least, data that may occur in the future may not be known now and moreover, may not be fully represented by familiar data from the past.

Unlike the traditional applications of computer programs, machine learning aims at automatically discovering the appropriate parameters of a selected algorithm, the implementation of which may already be available, but which needs its parameters adjusted to specific data.

1.5 Three Fundamental Learning Directions

Depending on information provided by collected teaching samples, the current machine learning sphere distinguishes three fundamental directions:

- Supervised learning
- Unsupervised learning
- Semi-supervised learning

1.5.1 Supervised Machine Learning

Known also as learning with a teacher, supervised machine learning relies on labeled training samples. Each sample $\mathbf{x}_i$ has its corresponding function value y_i :

$$y_i = f(\mathbf{x}_i).$$

If the function y_i is defined as non-numerical, its values are labels of classes. For example, books may be classified according to verbal evaluation by their readers who also assign a decision (label) $y_i \in \{$ interesting, tedious, neutral $\}$. A classification algorithm learns using the labeled samples. This learning process is called learning with a teacher because the trained algorithm has feedback based on the knowledge of the correct answers (labels). The feedback is provided by the 'teacher' that supervises the correctness of classifications and the 'learner' knows whether its answer was right or not so that it can improve its knowledge - appropriately adjusted parameters of the algorithm.

The regression methods for numeric functions are not discussed because they are not relevant here.

1.5.2 Unsupervised Machine Learning

Unlike supervised machine learning, the unsupervised version misses feedback from the absent teacher, thus the learners must learn themselves. The term 'absent teacher' means that the available training samples $\mathbf{x}_i$ lack their appropriate class labels, therefore it is not directly possible to reveal what is relevant for each class. In addition, it is not known what (and how many) classes exist for a given case. The task is now not only to learn how to classify correctly but also to which classes.

This branch of machine learning is called learning without a teacher and employs various clustering algorithms and methods. Based on the contents of $\mathbf{x}_i$'s, the clustering algorithms try to detect which samples are similar enough to establish their particular cluster. Then, each cluster represents a class. Usually, it is necessary to define (classes) in advance how many clusters should be generated; this is a frequent problem. Too few clusters can mean too vague knowledge mined from the data. The more clusters, the more specific each of them is; however, too many clusters lack the property of being generalized knowledge extracted from the samples - in the extreme case, each cluster contains only one case, which is not very useful knowledge.

There exist also certain clustering algorithms that are able to discover the number of convenient clusters; for example, the EM (expectation-maximization) algorithm is worth mentioning. For more details, see [112] chapter 8.5. EM assigns a probability distribution to each instance which indicates the probability of its belonging to each of the clusters. EM can decide how many clusters to create by cross validation, or it can be specified a priori how many clusters to generate.

The unsupervised learning belongs to more difficult and demanding learning methods because it must work with less information. However, clustering today constitutes one of the highly needed applications in the real world. An interested reader can find more information concerning the clustering methods in [142,230, 22, 254], and many others.

1.5.3 Semi-supervised Machine Learning

A modern machine learning branch is semi-supervised machine learning, which - as its name suggests - represents something in between the extreme cases of supervised and unsupervised learning methods. In semi-supervised machine learning, the learner is given only a few labeled training samples and many unlabeled ones.

The labeled samples provide some valuable initial information, which

makes it easier to separate (and label) unlabeled training samples so that the correct classes can be generated.

This task is typical for contemporary situations when it is relatively much easier to automatically collect and store a lot of unlabeled data than to laboriously label them all.

As a result of the use of modern information and communication technologies (the Internet, social networks), while there is an increasing amount of generally valuable (and not only) textual data on a daily basis their processing, for example from a semantic point of view, is now completely beyond human 'manual' processing and research.

A limited number of training samples gives a limited amount of information which can, however, be used advantageously for advanced text analysis. In any case, this limited information provides some guidance on how to deal with incomplete (unlabeled) data items. A teacher can prepare a small set of samples with answers (classification results), which may be used by the learner as a kind of clue to help solve the class assignment with respect to non-classified training items. At the same time, it makes it possible to better determine the number and types of suitable classification classes.

A good introduction with examples of several algorithms (Self-training, Cotraining, and some others) can be found in [1].

1.6 Big Data

One of the problems that we can experience in text mining is that there are too many data examples available. On the one hand, it can be considered positive in terms of sufficient coverage of data with training examples. However, on the other hand, it usually leads to great difficulties due to too high computational complexity (time and memory), which the available hardware may not be able to handle. This is a problem known as Big Data (which is, however, not discussed in this book; as an introduction, a reader may refer to, for example, [147, 152]).

For text mining, the mentioned big-data problem is often characteristic. The reason is that, thanks to broadly available world-wide web technologies like the Internet, textual data accumulate incredibly fast every day. Various social networks belong to most generous data providers and in principle, they represent a very valuable source of information not only for the contributors. Unfortunately, it is not known how to define a mathematical function, which would be able to process that natural-language based textual data so that the

data may be properly analyzed from its semantic point of view like human researchers can. Because of the huge amount of the data, it is necessary to use informatics and computers, which is where machine learning helps.

1.7 About This Book

Today, a lot of good publications and books are already available for familiarization with machine learning, for example [199, 151, 187, 261]. Tens of books from various publishers deal with many different problem areas from the machine learning point of view, and a reader will, undoubtedly be able to find what her or his specific interests need. Our goal is, therefore, not to teach machinelearning in general. Instead, we would like to show how machine learning can be used to solve practical tasks in a specific domain, which is text data analysis.

In order to obtain some real results and enable readers to implement some of the described procedures, we decided to use the R programming language. R is a great statistical software package with many advantages. It is free, it is interpreted (the same code can be run on many platforms), and provides an extensive functionality contained in many available packages. It is used by many researchers and practitioners all over the world. The solutions of problems using R are, however, often not suitable for large scale problems because R works only with data that fit to the memory of a computer and is often slow (the reduced speed is also because it is very flexible through dynamic typing).

In R and its available libraries, many machine learning algorithms are already implemented. It is, therefore, necessary to provide them with prepared data in a desired format, get the results, and display, visualize, or evaluate them. All these steps are combined in computer programs (scripts) that are run on an R interpreter.

In individual chapters dealing with typical text mining tasks, the code that is used to demonstrate the algorithms and solve some practical problems is also presented. To enable readers to understand the code, some of the R basics are included in the book too. We do not try to explain all aspects of the language, all available algorithm implementations, their possible parameters, inputs, and outputs, but only what is necessary for our specific examples. All other details can always be found in the newest description (manual) of the used R package.

Chapter 2

Introduction to R

R is a high-level programming language and environment for statistical computing and graphics. For many years, R has belonged to the most popular programming languages, especially in the machine learning community. It is primarily focused on statistical operations, working with vectors and matrices, creating statistical models on a few lines, etc. [209].

R provides facilities for effective data handling and storage, operators for calculations on multidimensional data, a collection of tools for data analysis and graphical facilities for data analysis and display. It also includes a simple and effective programming language which includes all common elements of programming languages, such as branchings, loops, user-defined functions, or input and output facilities [117]. The capabilities of R language can be augmented with compiled code written in C, C++, or Fortran [80].

The language, created by Ross Ihaka and Robert Gentleman at the University of Auckland more than twenty years ago, has derived its syntax, semantics, and implementation from two other older languages - S and Scheme [125]. R is an open-source project, which means that you can see how the functionality is implemented and modify what does not work exactly according to your requirements [203]. R is very popular in science, engineering, and education, with many active communities. The R Foundation supports the R project and its development, organizes conferences, and patronizes the R Journal (https://journal.rproject.org).

This book does not want to provide exhaustive information on programming in R. The language is used here as one of many options with the aim of demonstrating the principles of machine learning algorithms and getting real results from analyzing texts. An interested reader can refer to an endless resource of books (e.g., The R Primer by C. T. Ekstrom { CRC Press }, R in a Nutshell: A

Desktop Quick Reference by J. Adler {O'Reilly}, The R Book by M. J. Crawley {Wiley}, or The Art of R Programming: A Tour of Statistical Software Design by N. Matloff {No Starch Press}. Probably the most compete information can be found in The R Manuals available at https://cran.r-project.org/manuals.html.

Some of the tasks can be achieved with core R functions and capabilities, some other algorithms are implemented by various packages available in the Comprehensive *R* Archive Network (CRAN). The archive currently contains almost 14,000 packages solving problems from various domains and uploaded by many authors. To solve one specific task, multiple packages are often available. In this book, we do not consider all possible options as the main focus is not on R but the machine learning algorithms. Also only the necessary functionality (only some functions and their parameters) of the packages is used, which is sometimes only a small fraction of the package capabilities. The complete functionality and parametrization of algorithms can be always found in the package documentation.

2.1 Installing R

We assume that you have a PC or an Apple Mac and you want to install R on the hard disc and that you have access to the Internet. The R language is available as free software under the terms of the Free Software Foundation's GNU General Public License (https://www.gnu.org) in source code form at https://www.rproject.org. It can be compiled and run on a wide variety of UNIX platforms, as well as on Windows and Mac OS X.

In order to get a current version of the language, go to the CRAN site (https://cran.r-project.org/mirrors.html) and choose your preferred CRAN mirror. Then, you can decide the platform (Linux, Max OS X, or Windows) for which to obtain a precompiled binary distribution. You can also get the source code for all platforms (see Figure 2.1).

In our example, a version for the Windows operating system will be installed. The installation process for all other supported systems is described on the website of the R project.

There exist a few possibilities of the R download for Windows. The base sub-directory is preferred for the first installation.

While older distributions are still accessible, the newest distribution is recommended for downloading. The executable distribution file (e.g., R-3.5.1-win.exe) can be easily downloaded and run. *R* will be installed in an ordinary

process with the assistance of an installation wizard.

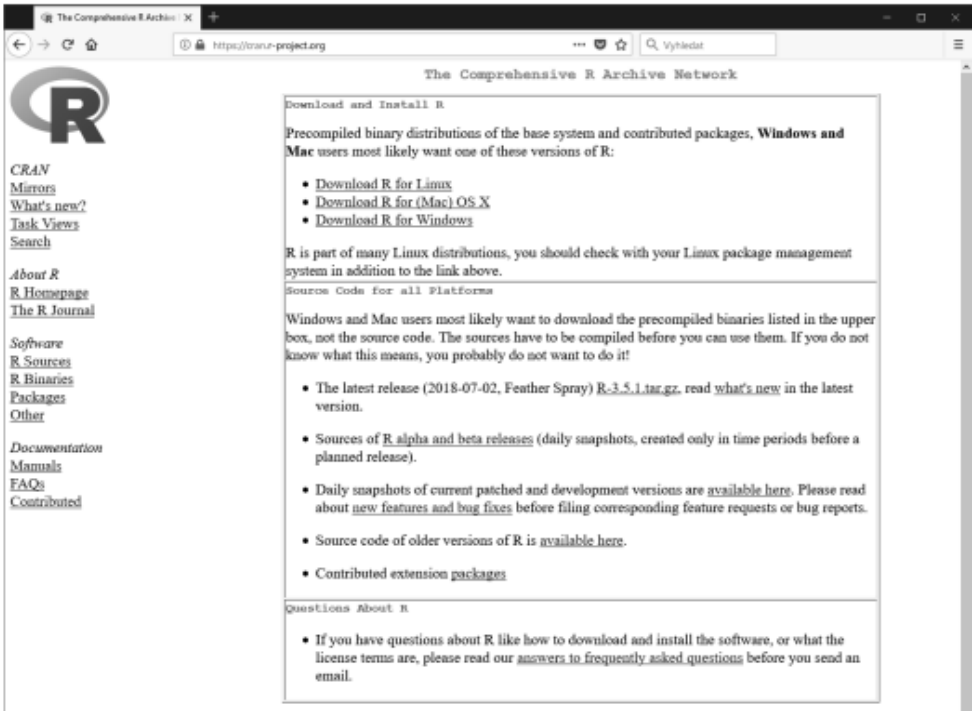


Figure 2.1: The home page of the R project.

2.2 Running R

To run R under Windows, just click on the R icon. A graphical user interface, as in Figure 2.2, will be displayed.

Inside the R graphical user interface, you can see a menu, toolbar, and R Console. In the console, below the header (a version of R and some initial information about the system) you will see a line with a > symbol on the left. This is called a prompt and this is where you type your commands. Try to enter citation() to see how to cite the R software in your work.

An easier way to use R is by using some of the integrated development environments (IDE). A list of some R IDEs includes:

- R Studio,
- Visual Studio for R,

- Eclipse,
- Red-R,
- Rattle,
- and many others.

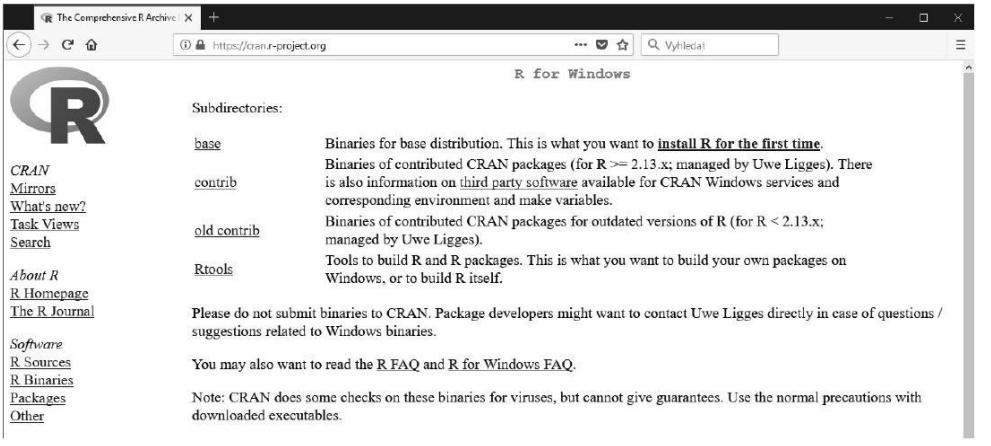


Figure 2.2: Selecting a subdirectory for R installation.

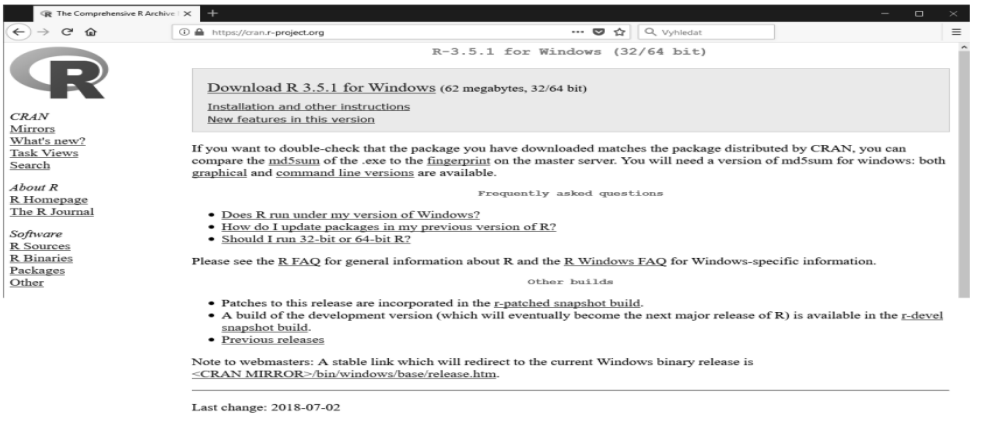


Figure 2.3: Getting the newest R package.

RStudio has been voted the most popular R IDE by the readers of KDnuggets[1]. Thus, in the following text, we are using RStudio for the examples.

[1] `https://www.kdnuggets.com/polls/2011/r-gui-used.html`

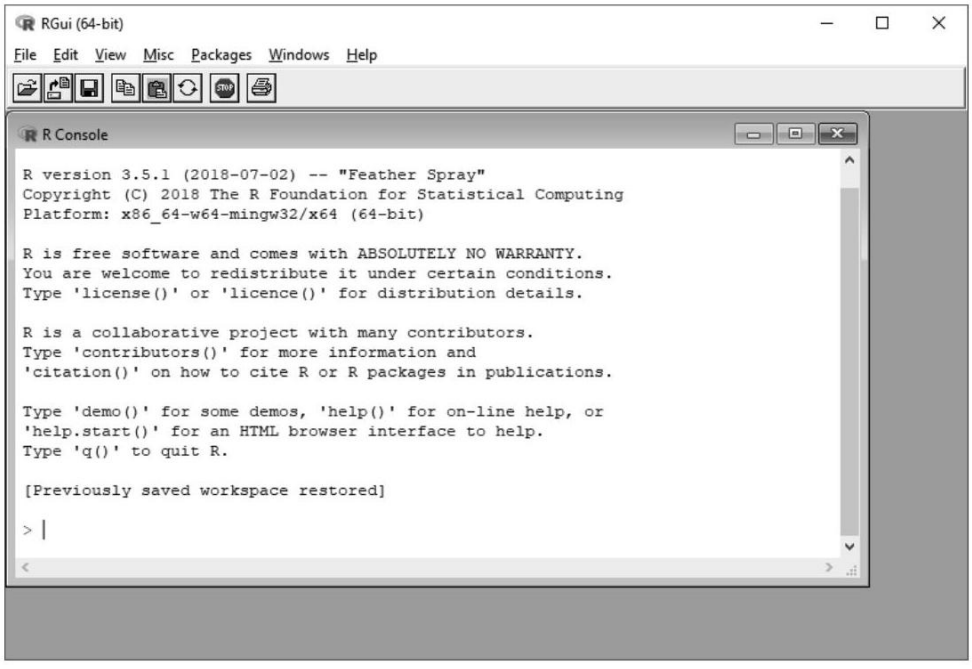


Figure 2.4: The graphical user interface of the R system on Windows.

2.3 RStudio

RStudio is an IDE for R available in open source and commercial editions. It enables syntax highlighting, code completion, and smart indentation when writing the code that can be executed directly from the source editor. It facilitates the work of programmers by providing facilities like jumping to function definitions, integrated documentation and help, managing multiple working directories, multiple-file editing, interactive debugger for diagnosing and fixing errors, and package development tools.

RStudio can be run on various operating systems like Windows, Mac OS X , and Linux. It is a very useful and powerful tool for programming in R . We suggest that readers use RStudio when learning from this book. To get RStudio, go to https://www.rstudio.com/ and follow the instructions.

On launching RStudio, the following panes are displayed (see Figure 2.5):

- Console where R commands/scripts can be executed and Terminal allowing the access to the system shell (top left-hand corner of the screen)

- Environment and History, and Connections - information about environment, history of commands, and connections to external data sources (top right-hand corner of the screen)

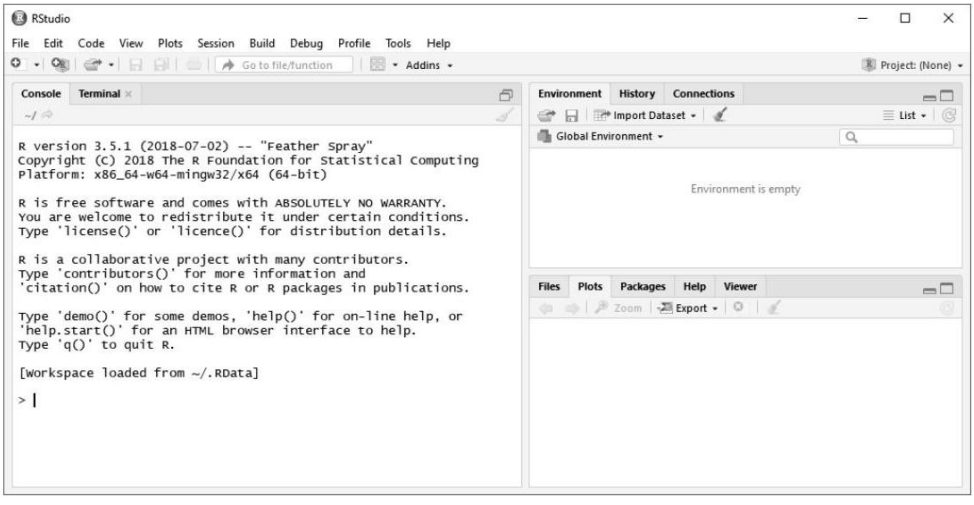


Figure 2.5: The graphical user interface of RStudio.

- Files in the current working directory, installed R packages, plots, a help display screen, and a viewer of local web content (bottom right of the screen).

A new script, i.e., a code that can be saved on a disk and later reopened or executed, can be created by choosing File / New File / RScript in the main menu. A new empty editor window, ready for editing, will appear.

It is always good practice to choose a working directory when programming in RStudio. The working directory is a folder where you store your R code, figures, etc. To set the working directory, go to Session / Set Working Directory / Choose Directory in the main menu.

2.3.1 Projects

RStudio provides a very useful feature known as Projects to organise your work into meaningful contexts. A project puts together R scripts, input data, analytical results, or plots, and has its own working directory, history, or workspace.

A new project can be created by selection of File / New Project in the main menu. A project can be started in a new directory, associated with an

existing working directory, or cloned from a version control (Git or Subversion) repository.

Select the New Directory option, then New Project, choose a name for your project directory, and select its location. A new directory will be created, together with a project file (with the .Rproj extension) and a hidden directory .Rproj.user with project-specific temporary files project. The will be loaded in RStudio and its icon displayed to the right of the toolbar. RStudio will also automatically set the working directory to the project directory.

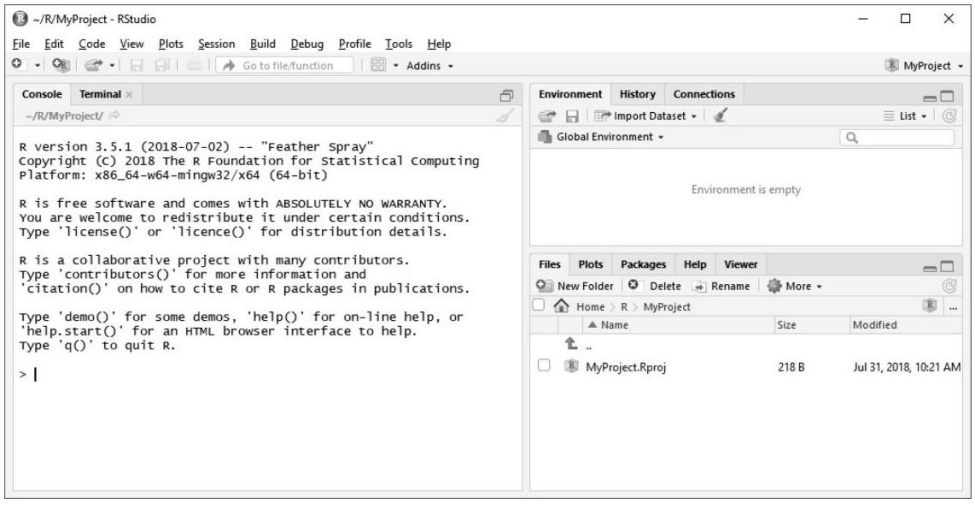


Figure 2.6: A new project created in RStudio.

2.3.2 Getting Help

Help pages for a desired object (e.g., operator or function) are invoked by the help() function. The name of an object for which help should be displayed is passed as a string parameter (enclosed in single or double quotes). However, a shortcut ? is often used (quotation marks not necessary). For example, typing ?sin opens a help page containing a documentation for the sin() function.

If you do not remember the precise name of the object but know the domain to which it is related (e.g., trigonometric functions in this case) use the help.search() function. A string passed to the function will be searched for in the documentation and the possibilities associated with this query will be displayed. The same can be achieved with the ?? shorthand (e.g., ??trigonometric or ??"trigonometric functions").

Another way to get help in RStudio is to select Help / R Help in the main menu or by going to the Help pane and browsing the manual pages or searching for a specific answer using the search capability.

To see some examples related to, for instance, a function, use the example() function, where the appropriate name is passed as a parameter (e.g., example(help) or example('+')).

2.4 Writng and Executing Commands

When R is ready to accept commands (expressions, consisting of objects and functions) from a user, the R console displays a > prompt. After it receives a command (something is typed on the keyboard and Enter is pressed) R tries to execute it. Subsequently, a result is shown [2] in case of a successful execution and a new > prompt is displayed again.

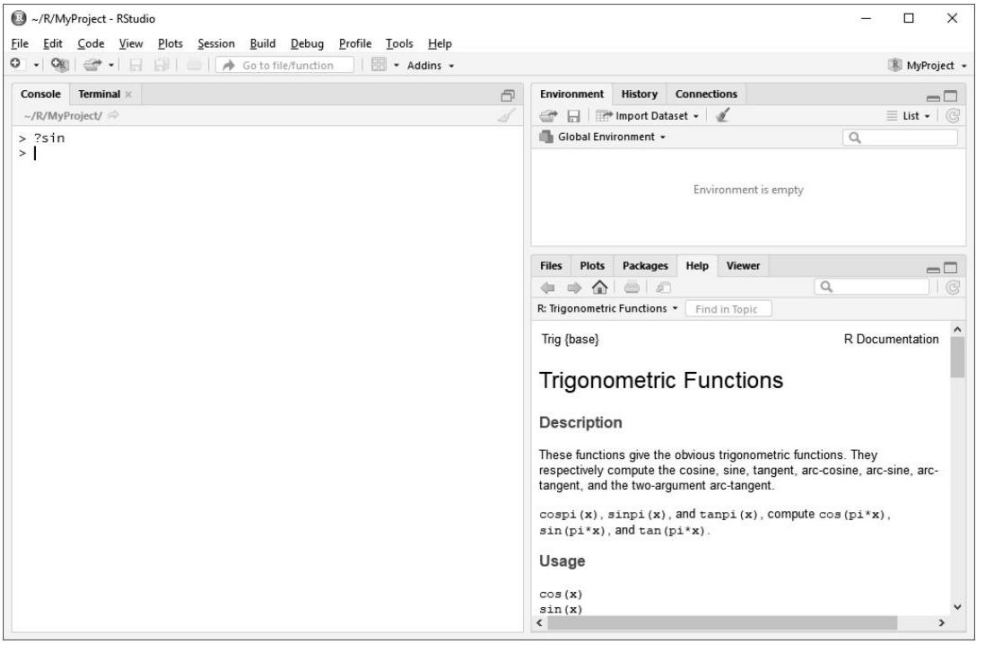


Figure 2.7: Displaying a help for an existing function.

[2] The number in square brackets is related to numbering the elements of the output and can be ignored. Here, for example, the output is a vector and the row starts with its first element.

```
> 3+8/4 # after writing the expression, Enter was pressed
[1] 5
> "Hello, World!"
[1] "Hello, World!"
>
```

When the command is not complete (for example, a closing parenthesis, quote, or operand is missing) R will submit a request to finish it. The console will now display a + prompt. The user can complete the command and the result will then be displayed. To finish writing the command when the + prompt is displayed, just press *Esc*.

```
> 5+ # Enter was pressed here
+ 6 # Enter was pressed here
[1] 11
> 5+ # Enter was pressed here
+ # Esc was pressed here

>
```

Two or more expressions can be placed on a single line. In that case they must be separated by semicolons with optional white spaces:

```
> 1+2;2-3; 4/5
[1] 3
[1] -1
[1] 0.8
```

To complete a piece of code, press Tab. A list of possible completion options (together with their short descriptions when applicable) that fit the given part of code is displayed (RStudio analyzes the code to suggest only meaningful options). By selecting one of them, the selected piece of text will be inserted to a command. If there exists only one possibility of what can be inserted no options are displayed and the code is completed immediately.

By pressing the up and down arrow keys, previously entered commands can be accessed. Pressing *Ctrl+Up* displays the list of all commands. They are also accessible in the History pane of RStudio.

Comments are useful pieces of the code that help programmers explain what various part of the code mean. They are completely ignored by the R interpreter. A comment starts with the symbol #. The subsequent characters until the end of line are the comment. R does not allow comments running into multiple lines. To overcome this problem, a quoted string occupying more than one line can be used. The string will be evaluated as normal but will not perform any action.

2.5 Variables and Data types

To store and access values in computer memory, so called 'objects' are used. Objects are data structures containing values of a certain type. Every such a structure might have a class attribute, which determines the class of the object, associated to it. If such an attribute does not exist, it will be computed from the type of the values stored in the object.

There exist a few atomic (single-valued) data types (classes, returned by the class() function as mentioned below) that are typically used in machine learning:

- character,
- numeric,
- integer,
- logical.

The basic data structure in R, where the atomic values are combined, is a vector; there are no scalars. The other structures include matrices, arrays, factors, and data frames. A vector is an ordered collection of values of the same type. A matrix is a two dimensional structure and an array can have even more dimensions. Their values are always of the same type. Lists allow storing of values that are of different types. Factors look like strings but represent categorical values internally stored as integers. Data frames store data in a tabular format; their individual rows represent data records and the columns hold their properties (the values in one column is always of the same type).

Unlike in some other programming languages, no data types need to be declared for variables. The data type of an object is given by what it contains. One object can hold a number and later, for example, a string.

2.6 Objects in R

All code in R manipulates objects [4]. Object is just something that is represented in a computer, like a vector, list, or data frame. It has a class that defines what functions can be used together with the object and how will they work. The other object classes besides the ones related to storing data include, for example, the function or expression classes.

```
> class(1)
[1] "numeric"
> class("hello")
[1] "character"
> class(class)
[1] "function"
> class(expression(1+2))
[1] "expression"
```

The objects usually have a name (known as a symbol) so we can later manipulate them. They thus behave like variables in other programming languages. The set of symbols is called the environment (which is also an object). The list of all objects in the current environment is returned by the objects() function. A new environment is created when a new session in R is started. A new environment is also created in the body of a function. The parent environment of the function is then the environment in which the function was created (not necessarily the same as where the function is called).

Variable names in R are case-sensitive so, for example, x is not the same as X . Variable names can contain alphanumeric characters, periods, and underscores, but they cannot start with a number or an underscore or with a period followed by a digit.

A list of existing objects in a specified environment (or objects defined by a user or local names in a function when called without parameters) is accessible through the ls() or objects() functions.

Variables can be easily removed using the remove() function (or using a shortcut rm()):

```
> x <- 1
> x
```

```
[1] 1
> remove(x)
> x
Error: object 'x' not found
```

Objects can have properties known as attributes. The list of attributes of an object is returned by the attributes() function. The attributes that are common to many object types include, for example, class - the class of an object, dim - dimensions of an object, names - the names of data elements (e.g., in a vector), levels - levels of a factor, among others.

To access or modify the value of an attribute, we usually use a function that has the same name as the attribute.

```
> x <- c(first=1, second=2, third=3)
> x
    first second third
        1 2 3
> attributes(x)
$'names'
[1] "first" "second" "third"
> names(x)
[1] "first" "second" "third"
> names(x) <- c("1st", "2nd", "3rd")
> x
1st 2nd 3rd
    1 2 3
> names(x)
[1] "1st" "2nd" "3rd"
> attributes(x)
$'names'
[1] "1st" "2nd" "3rd"
```

There are several so called *generic functions*. Their names are shared across different classes and they implement different algorithms depending on the class (they enable polymorphism). There is, for example, a difference when printing a number and a character string, where the latter requires double quotes to appear around the value.

There are also many functions that enable the examination of the features of different objects, for example:

- `class()` - the class of an object (a vector of names of classes an object inherits from); if not known, generic classes, like matrix, array, function, numeric, or the result of typeof() are returned
- `typeof ()` - the internal type of data stored in an object (e.g., integer or logical)
- `length()` - the length of an object, e.g., the number of vector elements
- `attributes()` - returns some metadata related to an object, like the dimensions of a matrix
- `str()` - compactly displays the internal structure of an object
- `exists()` - gives information about the existence of an object with a given name
- `get ()` - returns an object with a name passed as the parameter
- `print()`-prints the values of its arguments
- `summary()` - gives some basic information about a data structure, for example, the minimum, maximum, mean, median, first quartile, and third quartile for numeric vectors or the levels and numbers of elements for each level in the case of factors.

To find out whether an object is of a given class, the is() function returning a logical value can be used:

```
> x <- 1
> is(x,"numeric")
[1] TRUE
> is(x,"character")
[1] FALSE
```

Many common classes have their own functions which can answer the same question, for instance is.numeric(), is.matrix(), is.factor() etc.

To change the class of an object (which is known as casting [58]), a class can be directly assigned to the object:

```
> x <- 1
> x
[1] 1
```

```
> class(x) <- "character"
> x
[1] "1"
```

This is, however, not recommended. Instead, a set of functions with the names starting with as. that are corresponding to the is.* functions is available and should be used.

```
> x <- 1
> x <- as.character(x)
> x
[1] "1"
```

2.6.1 Assignment

The assignment operator <- points to the object receiving the value of the expression stated on the right side. To create an object with value 10 we can write

```
> x <- 10
> x
[1] 10
```

In an assignment, the objects are usually immutable which means that their values are copied instead of assigning just a reference to them, as shown in the following example:

```
> a <- 1
> b <- a
> c <- a # both 'b' and 'c' contain the value of 'a'
> a
[1] 1
> b
[1] 1
> c
```

```
[1] 1 # both 'b' and 'c' have the same value 1
> b <- 2
> b
[1] 2
> c
[1] 1 # the value of 'c' has not changed
> a <- 2
> c
[1] 1 # the value of 'c' still has not changed
```

There are a few other alternatives besides $<-$, such as $=, ->, \ll -$, and $- \gg$. It is, however, recommended that <- is used, because it always means assignment. Assignment of a value to a variable can be also achieved with using the assign() function (a variable name is passed as a character string):

```
> assign("x", 10)
```

2.6.2 Logical values

Logical values can either be TRUE (when used as numbers, they are treated as 1) or FALSE (numerically 0). Shortcuts T and F can be used instead which is, however, not recommended since these names can be also used as variable names and thus easily overwritten.

```
> class(TRUE)
[1] "logical"
> 2*TRUE
[1] 2
> 2*FALSE
[1] 0
> T
[1] TRUE
> T == TRUE
[1] TRUE
> T <- 0
> T == TRUE
```

```
[1] FALSE
```

2.6.3 Numbers

Decimal values that are called numeric in R are the default computational data type. They include integers and decimals, positive, negative, and zero. The values are represented in double precision floating point. Integers which hold only whole numbers need to have an *L* appended at the end of the value.

```
> class(1)
[1] "numeric"
> class(1L)
[1] "integer"
> class(1.5)
[1] "numeric"
```

To create an integer variable in *R*, we can also call the as .integer function.

```
> class(1)
[1] "numeric"
> class(as.integer(1))
[1] "integer"
> class(as.integer(1.8))
[1] "integer"
> x <- as.integer(1.8)
> x
[1] 1
> class(x)
[1] "integer"
```

Optionally, numbers can be written using the semi-logarithmic notation.

```
> 1e2
[1] 100
> class(1e2)
```

```
[1] "numeric"
> 1e-2
[1] 0.01
```

You may also specify the value in the hexadecimal notation by prefixing the value with 0x.

```
> 0x1
[1] 1
> 0xff
[1] 255
> class(0xff)
[1] "numeric"
```

R can also work with complex numbers (written for example, as $1 + 1i$) but they are not very common in machine learning so far.

There are a number of built-in functions for working with numbers. They represent typical operations, like the absolute value (abs()), square root (sqrt()), logarithm (()), exponentiation (exp()), different forms of rounding (floor(), ceiling(), trunc(), round(), signif()), trigonometric and inverse trigonometric functions (sin(), cos(), tan(), asin(), acos(), atan()). Most of these functions work with one numeric vector (and are applied to its components). Some other functions require more arguments, for instance log(), which can take a base for the logarithm. Trigonometric functions in R measure angles in radians. The whole circle is 2π radians, a right angle is $\pi/2$ radians. The value of the Ludolphian number π is known as pi in R.

2.6.4 Character Strings

A data type enabling working with text data is character. Strings constants are enclosed between a pair of single or double quotes. Double quotes are preferred as strings are also quoted by double quotes on the output (to not quote them on the output, use the noquote() function). When a single or double quote is to appear in a string, the string might be enclosed between a pair of the other quote type:

```
> "This book is published by 'CRC Press'"
[1] "This book is published by 'CRC Press'"
> 'This book is published by "CRC Press"'
[1] "This book is published by \"CRC Press\""
```

The quotes can also be escaped by a backslash so they are not considered terminating quotes:

```
"This book is published by \"CRC Press\""
[1] "This book is published by \"CRC Press\"'\n"
```

There are other symbols which can be inserted to strings using escaping characters by a backslash, examples being \n (newline), \r (carriage return), \t (tabulator), \b (backspace), \\ (backslash), \nnn (character with the given octal code consisting of 1 to 3 digits), \xnn (a character with the given hexadecimal code consisting of 1 or 2 digits), \ unnnn (a Unicode character with the given code consisting of 1 to 4 hexadecimal digits), etc.

Since string data can be a part of one of the structures where R stores data, it is necessary to understand how they behave [237]:

- vector, matrix - if one or more elements are strings, all data in a vector or matrix will be treated as strings
- data frames - strings are converted to factors by default; to turn this behaviour off, the stringsAsFactors parameter should be set to FALSE in the data.frame() function while creating a data frame object
- list - all elements maintain their types

There are many functions which can be used to handle string data. Here are a few typical examples:

- nchar () - returns number of characters of a string [3]
- tolower (), toupper() - returns a string converted to lower/upper case
- substr(), substring() - returns or replaces substrings in character vectors
- strsplit() - splits a string to substring according to the match of one or more regular expressions

[3] The function length(), that is, in many programming languages, used to give a string length, returns the number of elements in a data structure in R .

2.6.5 Special values

Missing values are represented by a constant NA (NA stands for Not Available). The is.na() or complete.cases() functions can be used to detect NA values in data structures.

```
> x <- c(1, 2, 3, NA, 5)
> x
[1] 1 2 3 NA 5
> is.na(x)
[1] FALSE FALSE FALSE TRUE FALSE
> complete.cases(x)
[1] TRUE TRUE TRUE FALSE TRUE
```

Calculations can sometimes lead to results that are either too big or too small to be stored in a memory. Very big (i.e., positive numbers) are represented by the Inf constant and very small (i.e., negative numbers) are represented by the -Inf constant.

```
> 2^1000
[1] 1.071509e+301
> 2^10000
[1] Inf
> -2^10000
[1] -Inf
```

To detect finite and infinite numbers, the is.finite() and is.infinite() functions can be used:

```
> x <- c(1, 1/0, -2^1000)
> x
[1] 1.000000e+00 Inf -1.071509e+301
> is.finite(x)
[1] TRUE FALSE TRUE
```

Other calculations can lead to results that cannot be represented as numbers. They are known as NaN (NaN stands for Not a Number). Basically all mathematical functions in R are supposed to work properly with NaN as input or output. To test whether a value is NaN , the is.nan() function is used.

```
> Inf/Inf
[1] NaN
> 0/0
[1] NaN
> Inf-Inf
[1] NaN
```

NULL is an object that represents an undefined object or a list with zero length (all NULLs point to the same object). It is not the same as missing value. It is often used as a parameter of a function when no value for that parameter should be passed or returned by a function when the value is undefined.

The function as. null() ignores its argument and returns NULL; is. null() tests whether the argument has an undefined value.

Dates are internally stored as numbers of days or seconds (depending on the class) since 1st January 1970. There are two date classes, Data, storing only information about days, and POSIXct, which also contains information about time. To create a date value, functions asData() or as.POSIXct() are used to convert strings to dates.

```
> date1 <- as.Date('2018-01-01')
> date1
[1] "2018-01-01"
> class(date1)
[1] "Date"
> typeof(date2)
[1] "double"
> date2 <- as.POSIXct('2018-01-01 10:20:30')
> date2
[1] "2018-01-01 10:20:30 CET"
> class(date2)
```

```
[1] "POSIXct" "POSIXt"
> typeof(date2)
[1] "double"
```

2.7 Functions

Functions are objects that accept a set of arguments, somehow process them, and return a value. They may, therefore, be used as parts of expressions.

Some functions are associated to some classes. They are known as methods. Unlike in other languages (like Java), not all functions are closely tied to a particular class in R [4].

Some of the methods have the same name in different classes. They are called generic functions. They enable the use of the same code for working with objects of different classes. The most visible generic function is print() which is automatically called after commands that are evaluated by R return any object. The objects are somehow printed depending on their type (e.g., a vector is printed as a list of whitespace separated values on a row, a matrix is printed as M rows and N columns).

There exist a large number of built-in functions from many categories which are readily available for use. They include functions enabling working with objects of different types of data (functions for numbers, character string, factors etc.), mathematical, statistical or optimization functions, functions for generating graphics, etc.

Other functions become available after a package containing their definitions is loaded.

Sometimes, a programmer might need a function that is not available and needs to create a new one. In that case, the algorithm the function implements needs to be created and the interface of the function must be specified. The interface includes the parameters and the value to be returned by the function. The body of the function is a code that usually processes input parameters to produce a return value. A function often has a name, which means that the function is stored in an object with that name.

To define a function, the function statement is called. Here, the parameters (arguments) of a function are defined, together with the function body.

The section with argument definitions contains the names of arguments so that their names can be used in the function body. The body of a function contains some statements to be executed. If more than one statement is

contained in the body, they need to be enclosed within curly brackets. The return value, which can be an object of any type, is the value of the last evaluated expression. To explicitly return a value, the return() function is used.

```
> XtoN <- function(x, n) {
+ return(x^n)
+ }
> XtoN(2, 3)
[1] 8
```

The parameters can also have default values. The default values are used when real parameters are not supplied. Such parameters are also treated as optional.

```
> XtoN <- function(x, n=1) {
+ return(x^n)
+ }
> XtoN(2)
[1] 2
```

Parameters in R are often of a variable length. The list of such parameters is known as an object with a special name . . . (available in a function body).

```
> sum_all <- function(...) {
+ s <- 0;
+ for (x in list(...)) {
+ s <- s+x
+ }
+ return(s)
+ }
> sum_all(1, 2, 3, 4, 5)
[1] 15
```

Modifying a parameter inside a function does not change the value of the real parameter. The initial value of the parameter will be the one of the actual parameter, but its subsequent modifications will not affect the actual argument. In addition, a variable appearing within a function that has the same name as a variable that is global to it will be treated as local. This means that changing its value in a function will not affect the value of the global variable. Only its initial value will be that of the global [183].

To modify the value of an actual parameter or of a global variable, the superassignment operator «- is used.

```
> x <- 1
> add_one <- function(x) x <- x+1
> add_one(x)
> x
[1] 1
> add_one <- function(x) x <<- x+1
> add_one(x)
> x
[1] 2
```

Let's have a function XtoN() for calculating the value of base to the power of the exponent:

```
XtoN <- function(base, exponent) return(base^exponent)
```

When submitting actual parameters during when calling a function, the following possible methods, that are tried in the following order, can be used [213, 4]:

1. The parameters are submitted as pairs full_name=value. Then, the order of parameters does not matter.

```
> XtoN(base=2, exponent=3)
[1] 8
> XtoN(exponent=3, base=2)
[1] 8
```

2. The parameters are submitted as pairs name = value, where name can be only a part of a parameter name. The order of parameters is also not important.

```
> XtoN(exp=3, b=2)
[1] 8
```

3. The values are passed in the same order as in the definition of the function. The first parameter will be stored to x and the second to n .

```
> XtoN(2, 3)
[1] 8
```

Information about the parameters of a function can be obtained from the documentation or by calling the args() function:

```
> args(XtoN)
function (base, exponent)
NULL
> args(sum_all)
function (...)
NULL
```

Functions can be also passed as arguments to other functions. In this case, they are often anonymous functions, which means that they do not have a name.

```
> myfunction <- function(x, f) { return(f(x)) }
> myfunction(c(5, 2, 3, 4, 8), min)
[1] 2
> myfunction(c(5, 2, 3, 4, 8), max)
[1] 8
> myfunction(c(5, 2, 3, 4, 8),
+ function (x) {return(length(x))}
+ )
[1] 5
```

2.8 Operators

An operator is a function working with one or two parameters which do not require writing parentheses. Some of the basic operators, which have not been discussed in much detail so far, are listed in Table 2.1. A user can also write his own binary operators. They are named %operator_name% and are implemented through functions.

Table 2.1: Some commonly used operators in R.

	Arithmetic Operators
+	addition
-	subtraction (binary) or arithmetic negation (unary)
*	multiplication
/	division
%%	the modulo operation (the remainder after division of one number by another)
%/%	integer division
^	exponentiation (raising to a power)
	Relational Operators
<, <=	less than, less or equal
>, >=	greater than, greater or equal
==	equal to
!=	not equal to
	Logical Operators
!	negation
|	logical disjunction (OR), processes all vector elements
||	logical disjunction (OR), works only with the first vector element
&	logical conjunction (AND), processes all vector elements
&&	logical conjunction (AND), works only with the first vector element
	Colon Operator
:	sequence of numeric values

In order to write expressions so that they return a correct value, the precedence (priority) of operators is important. The following list contains a complete set of operators in R, placed according to their priority in descending order [260]:

- function calls and groupings (({)
- access variables in a namespace (:: :::)
- component / slot extraction ($ @)
- indexing ([[[)
- exponentiation (right to left) (^)
- unary plus and minus (+ -)
- sequence operator (:)

- special operators (%any%, including % % and %/%)
- multiplication, division (*/)
- addition, subtraction (+ -)
- comparison (<><=>=== !=)
- negation (!)
- logical conjunction (& &&)
- logical disjunction (|||)
- separating the left and right-hand sides in a model formula (~)
- rightwards assignment (-> ->>)
- assignment (right to left) (<− ≪ −)
- assignment (right to left) (=)
- help (?)

2.9 Vectors

The simplest data structure in R is a vector. It is an ordered collection of values of the same type (they are known as atomic structures). We can therefore distinguish, for example, numeric (integer and double-precision), logical (containing the values TRUE, FALSE, and NA), and character vectors. Even a single number in an expression (like 1 + 2) is treated as a numeric vector with length 1 .

2.9.1 Creating Vectors

To create a vector, the function c() is typically used. The function takes its parameters and creates a vector from them. If the parameter values are of different types they are coerced into one type (c might thus stand for coerce).

```
> x <- c(1, 2, 3, 4, 5)
> x
[1] 1 2 3 4 5
> x <- c(1, 2, 3.5)
> x
[1] 1.0 2.0 3.5
> x <- c(1, 2, 3, "a", "b")
> x
[1] "1" "2" "3" "a" "b"
> x <- c(1, 2, 3, TRUE)
```

```
> x
[1] 1 2 3 1
```

If you forget to write the name of function c() , which is a common problem in R beginners, you will get an error.

```
x <- (1, 2, 3)
Error: unexpected ',' in "x <- (1,"
> cummin(1, 2, 3)
Error in cummin(1, 2, 3) :
    3 arguments passed to 'cummin' which requires 1
```

A vector can be also created using the vector() function. The first parameter (mode) specifies the type of the vector (numeric, logical, character, list, or expression). The second parameter (length) defines the number of elements to be created. All elements will have a value that means nothing (0 for numbers, "" for characters, FALSE for logical etc.).

The colon operator (:) can be used to create numeric sequences too. It requires two values. If the first is less than the second, an ascending sequence of numbers starting with the first value and ending with the second value with step 1 will be created. If the first value is greater than the second, a descending sequence will be produced.

```
> x <- 1:5
> x
[1] 1 2 3 4 5
> x <- 5:1
> x
[1] 5 4 3 2 1
```

A more flexible way to create sequences is using the seq() function. It accepts a few parameters (not all of them are mandatory): from - the starting value of a sequence, to - the end value of the sequence, by - an increment of the sequence, length.out - a desired length of the sequence, along.with - an object from which the length of the sequence is taken.

```
> x <- seq(1, 5)
> x
[1] 12345
> x <- seq(-1, 1, by=0.3)
> x
[1] -1.0 -0.7-0.4-0.1 0.2 0.5 0.8
> x <- seq(-1, by=0.3, length=3)
> x
> y <- 1:3
> x <- seq(1, 5, along.with=j)
> x
[1] 135
```

Another useful function is rep(). It replicates an object according to the passed arguments which include: times - how many times the values should be repeated, length.out - what should be the length of the sequence, and each how many times each element will be repeated.

```
> x <- c(1, 2, 3)
> rep(x)
[1] 1 2 3
> rep(x, times=2)
[1] 123123
> rep(x, times=2, length.out=10)
[1] 1231231231
> rep(x, length.out=10)
[1] 1231231231
> rep(x, length.out=10, each=4)
[1] 1 1 1 1 2 2 2 2 3 3
```

2.9.2 Naming Vector Elements

In some situations, it is more convenient to access vector values using some names. They can usually be remembered more easily than numbers (positions).

Fortunately, R provides a mechanism of naming vector elements. The names can be defined at the moment of creating a vector. The name (if it

looks like a regular name, it does not have to be quoted), together with a equal sign, precedes a value. On the output, the names will be displayed above the values of the vector.

```
> x <- c(first=1, second=2, third=3)
> x
    first second third
        1      2     3
```

Not all values need to have names.

```
> x <- c(first=1, 2, third=3)
> x
    first   third
        1 2     3
```

The names of a list are returned by the names() function. When a vector has no element names, NULL is returned.

```
> x <- c(first=1, second=2, third=3)
> names(x)
[1] "first" "second" "third"
```

The names can be at any time modified using the names attribute of a vector (through the names() function).

```
> day_names <- c('Mon', 'Tue', 'Wed', 'Thu', 'Fri',
+ 'Sat','Sun')
> x <- 1:7
> names(x) <- day_names
> x
Mon Tue Wed Thu Fri Sat Sun
  1   2   3   4   5   6   7
```

When enough values are not provided for the names, their value will be NA. An empty name is simply an empty string.

```
> x <- 1:5
> names(x) <- c("a", "b", "", "d")
> x
    a b   d <NA>
    1 2 3 4 5
```

2.9.3 Operations with Vectors

When vectors are used in arithmetic expressions, the operations are performed on the level of individual vector elements. The result has the same length as the longest vector. The values of the shorter vector are repeated (recycled) so there are enough values as needed for all the operations.

```
> x <- c(1, 2, 3)
> 2*x
[1] 246
> x <- c(1, 1, 1, 1, 1, 1)
> y <- c(1, 2)
> x+y
[1] 2 3 2 3 2 3
```

There is a similar situation in the case of logical operations. The operation is performed with each element of a vector and the result will be a logical vector with the length of the original vector.

```
> x <- 1:5
> y <- x>=3
> y
[1] FALSE FALSE TRUE TRUE TRUE
> !y
[1] TRUE TRUE FALSE FALSE FALSE
```

There are many useful functions operating on vectors, like min() (finds the minimum), max() (finds the maximum), range() (finds both extremes), sum() (calculates the sum of values), prod() (calculates the product of all values), mean() (returns the mean), var() (calculates the variance), sort() (sorts the values), order() (returns the order of sorted values in the input), or length() (counts the elements of a vector).

```
> min(c(5, 3, 4, 9, 1, 2))
[1] 1
> max(c(5, 3, 4, 9, 1, 2))
[1] 9
> range(c(5, 3, 4, 9, 1, 2))
[1] 1 9
> sum(c(5, 3, 4, 9, 1, 2))
[1] 24
> prod(c(5, 3, 4, 9, 1, 2))
[1] 1080
> 5*3*4*9*1*2
[1] 1080
> mean(c(5, 3, 4, 9, 1, 2))
[1] 4
> var(c(5, 3, 4, 9, 1, 2))
[1] 8
> length(c(5, 3, 4, 9, 1, 2))
[1] 6
> sort(c(5, 3, 4, 9, 1, 2))
[1] 123459
> order(c(5, 3, 4, 9, 1, 2))
[1] 5 6 2 3 1 4
```

The vector length can be changed using the length attribute.

```
> x <- 1:10
> length(x) <- 5
> x
[1] 1 2 3 4 5
> x <- 1:10
```

```
> x
[1]   1   2   3   4   5   6   7   8   9   10
> length(x) <- 5
> x
[1] 12345
> length(x) <- 10
> x
[1] 1 2 3 4 5 NA NA NA NA NA
```

The paste() function takes an arbitrary number of character vectors and concatenates them, one by one, into one vector. All non-strings will be coerced
into character strings. The sep parameter is used to connect individual concatenated strings (a space is used by default). If the collapse parameter is provided, all strings from the resulting vector are concatenated to a single string where the value of the collapse parameter is used to join the elements.

```
> paste(c("a", "b", "c"), 1:3, c("x", "y", "z"))
[1] "a 1 x" "b 2 y" "c 3 z"
> paste(c("a", "b"), rep(1:5, each=2), sep="-")
[1] "a-1" "b-1" "a-2" "b-2" "a-3" "b-3" "a-4" "b-4"
[9] "a-5" "b-5"
> paste(c("a", "b", "c"), 1:3, sep="", collapse="+")
[1] "a1+b2+c3"
```

2.9.4 Acessing Vector Elements

Some of the elements of a vector may be accessed using an integer vector known as an index vector (non-integer values are converted to integers by truncating the values). The index vector, which is written into a pair of square brackets, contains the positions of elements that are to be processed. The elements are counted from 1.

The following example shows how to retrieve a vector member or several members.

```
> x <- 10:15
> x
[1] 10 11 12 13 14 15
> x[2]
[1] 11
x[2:4]
[1] 11 12 13
> x[2.4]
[1] 11
> x[2.7]
[1] 11
```

If an index is out of the range of the vector, a missing value (NA) will be returned.

```
x[c(1,20)]
[1] 10 NA
```

Using a logical vector as the index means that only the elements which correspond to the positions with TRUE values of the index will be included in the output. Both vectors should normally have the same length. If the index vector is shorter, its elements will be recycled.

```
> x[c(TRUE, FALSE, FALSE, TRUE)]
[1] 10 13 14
```

If the index values are negative they are understood as positions of vector elements to be excluded.

```
> x
[1] 10 11 12 13 14 15
> x[-1]
[1] 11 12 13 14 15
> x[-seq(from=2, by=2, along.with=x)]
[1] 10 12 14
```

When the elements of a vector have names and the index vector contains strings, elements corresponding to the names are returned. When no names exist or when a non-existing name is supplied the corresponding elements will have the NA value.

```
> x <- c(first=1, second=2, third=3)
> x["first"]
first
    1
> x["firs"]
<NA>
 NA
```

2.10 Matrices and Arrays

Matrices are two-dimensional objects that, similarly to vectors, contain values of the same type (numbers, characters, logical values). To create a matrix, the matrix() function might be used. A matrix is created from some object containing data (passed as the first parameter or a parameter named data) and is organized into two dimensions according to the parameters of the function. The
parameters include: nrow - the number of rows, ncol - the number of columns, byrow - when FALSE, which is a default option, the matrix is filled by columns, otherwise by rows, and dimnames - a list with the names assigned to the rows and columns.

```
> matrix(1:9, nrow=3) # same as matrix(data=1:9, nrow=3)
     [,1] [,2] [,3]
[1,]    1    4    7
[2,]    2    5    8
[3,]    3    6    9
> matrix(1:9, nrow=3, byrow=TRUE)
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
[3,]    7    8    9
```

```
> rnames <- c("x1", "x2", "x3")
> cnames <- c("y1", "y2", "y3")
> matrix(1:9, nrow=3, byrow=TRUE,
+ dimnames=list(rnames, cnames))
    y1 y2 y3
x1   1 2 3
x2   4 5 6
x3   7 8 9
```

Information about matrix dimensions can be obtained using the nrow, ncol, and dim functions.

```
> x <- matrix(1:10, nrow=5)
> nrow(x)
[1] 5
> ncol(x)
[1] 2
> dim(x)
[1] 5 2
> length(x)
[1] 10
```

To access individual elements of a matrix, two values of an index are needed. They are again written between a pair of square brackets. When accessing a position out of the range of the two indexes, an error is caused.

```
> x <- matrix(1:10, nrow=5, byrow=TRUE)
> x[2, 1]
[1] 3
> x[2,4]
Error in x[2, 4] : subscript out of bounds
```

The entire row of a matrix can be accessed by leaving the second index empty. When we want to access a column, the row index is not entered.

```
> x <- matrix(1:10, nrow=5, byrow=TRUE)
> x[2,]
[1] 34
> x[, 1]
[1] 1 3 5 7 9
```

When the indices for a row, a column, or both, contain a vector, more than one rows or columns are accessed at once. The same principles apply as when accessing vector elements (you might use, for example, a logical vector).

```
> x[c(1, 3, 5), ]
     [,1] [,2]
[1,] 1 2
[2,] 5 6
[3,] 9 10
> x[c(1, 3, 5), c(FALSE, TRUE)]
[1] 2 6 10
```

Arithmetic operations with matrices work element-wise.

```
> m1 <- matrix(rep(2, 4), nrow=2)
> m2 <- matrix(rep(3, 4), nrow=2)
> m1+m2
     [,1] [,2]
[1,] 5 5
[2,] 5 5
> m2-m1
     [,1] [,2]
[1,]   1    1
[2,]   1    1
> m2*m1
     [,1] [,2]
[1,] 6 6
[2,] 6 6
> m1/m2
         [,1] [,2]
[1,] 0.6666667 0.6666667
```

```
[2,] 0.6666667 0.6666667
> sin(m1)
         [,1] [,2]
[1,] 0.9092974 0.9092974
[2,] 0.9092974 0.9092974
```

An array is an object which can store data in more than two dimensions. In fact, arrays are very similar to matrices (arrays contain matrices - for example, a three dimensional array is like a "list" containing two dimensional matrices). When creating arrays or when accessing the elements of parts of arrays, we need more indices. An array is created using the array() function.

```
> x <- array(1:18, dim=c(3, 2, 3))
> x
, , 1
     [,1] [,2]
[1,] 1 4
[2,] 2 5
[3,] 3 6
, , 2
     [,1] [,2]
[1,] 7 10
[2,] 8 11
[3,] 9 12
, , 3
     [,1] [,2]
[1,] 13 16
[2,] 14 17
[3,] 15 18
> x[, , 1]
     [,1] [,2]
[1,] 1 4
[2,] 2 5
[3,] 3 6
> class(x[, , 1])
[1] "matrix"
> x[, 1, 1]
[1] 1 2 3
```

```
> x[1, 1, ]
[1] 1 7 13
```

2.11 Lists

A list is an object that can hold data elements of different types. It is, therefore, a heterogeneous structure that is very common in many situations. It can contain vectors, matrices, arrays, lists, or even functions. The list() function is used to create a list. A list is created from the objects that are passed as parameters similar to the case of vectors. Names can be assigned to individual elements during the creation of a list or later, using the names() function.

```
> person <- list("John", "Smith", c("Peter", "Paul"), TRUE)
> person
[[1]]
[1] "John"
[[2]]
[1] "Smith"
[[3]]
[1] "Peter" "Paul"
[[4]]
[1] TRUE
> person <- list(name="John", surname="Smith",
+ children=c("Peter", "Paul"),
+ insurance=TRUE)
> person
$'name'
[1] "John"
$surname
[1] "Smith"
$children
[1] "Peter" "Paul"
```

```
$insurance
[1] TRUE
```

The length of a list is the number of top-level elements.

```
> length(person)
[1] 4
```

Since lists can contain objects of many different types, they have no dimensions. It does not make sense either, to carry out arithmetic operations upon lists (e.g., sum two lists). The calculations can, of course, be made on the level of individual list elements.

To access list elements, the same way of indexing, as in the case of vectors, is used. We can use numeric (both positive and negative) or logical indexes or element names written within a pair of square brackets. The indexing operation using single square brackets always returns a list.

```
> person[1]
$'name'
[1] "John"
> person["children"]
$'children'
[1] "Peter" "Paul"
> class(person["insurance"])
[1] "list"
```

When we need to access the content of a list element, we write the number or name of the element between a pair of double square brackets.

```
> person[[2]]
[1] "Smith"
> person[["children"]]
[1] "Peter" "Paul"
> class(person[["insurance"]])
[1] "logical"
```

The same can be achieved using the notation of the dollar sign after a list name, followed by an element name. The advantage of this notation is that in many R environments, the item names are automatically filled and partial matches are also accepted.

```
> person$surname
[1] "Smith"
> person$child
[1] "Peter" "Paul"
```

2.12 Factors

Factors are used to represent data that can have a finite number of values (we can say that they are categorical or enumerated). A typical example is of class labels in a classification problem. The labels look like strings but, in fact, represent the categories of data. The different values of a factor are known as levels. To create a factor from a vector, the factor() function is used. The levels() function returns the levels of a factor.

```
> labels <- c("negative", "neutral", "positive",
+ "neutral", "positive")
> labels
[1] "negative" "neutral" "positive" "neutral" "positive"
> factor(labels)
[1] negative neutral positive neutral positive
Levels: negative neutral positive
> nlevels(factor(labels))
[1] 3
```

When creating a factor, the levels allowed can be supplied using the levels parameter; excluded levels are listed in the exclude parameter (other values will be transformed to NA). The level names can be defined with the labels parameter which also enables mapping multiple values to one level.

Internally, factors are stored as vectors of integer values where the corresponding labels are mapped to them. This is a great improvement in efficiency,

since storing integers is much more efficient than storing strings. Normally, the order of factor values is not important. In some situations, however, such an ordering makes sense (for example, positive is more than negative, February is less than April).

The order of levels is given by the order of values in the levels parameter of the factor() function (when not provided, the levels are not ordered and are displayed in an alphabetical order). Ordering is visualized in the output of a factor. The ordering can be later changed by assigning new levels to the levels() function or calling the factor() function with an existing factor as a parameter together with new levels.

```
> f <- factor(labels,
+ levels=c("positive", "neutral", "negative"),
+ ordered=TRUE)
> f
[1] negative neutral positive neutral positive
Levels: positive < neutral < negative
> f <- factor((f),levels=c("negative","neutral","positive"))
> # the same as above
> levels(f) <- c("negative", "neutral", "positive")
> f
[1] positive neutral negative neutral negative
Levels: negative < neutral < positive
```

Ordering factor levels is important when we want, for example, to compare factor values.

```
> f
[1] positive neutral negative neutral negative
Levels: negative < neutral < positive
> ff <- f
> ff <- factor(f, ordered=FALSE)
> ff
[1] positive neutral negative neutral negative
Levels: negative neutral positive
> f[1]<f[2]
[1] FALSE
```

```
> ff[1]<ff[2]
[1] NA
Warning message:
In Ops.factor(ff[1], ff[2]) : < not meaningful for factors
```

To get rid of unused factor levels (when there are more levels defined than in the data), the droplevels() function can be used.

To access individual factor elements, an index can be used in the same manner as in case of vectors (positive and negative integers, logical values, and names).

```
> f[1]
[1] positive
Levels: negative < neutral < positive
> f[c(-1, -3)]
[1] neutral neutral negative
Levels: negative < neutral < positive
> f[c(TRUE, FALSE, TRUE)]
[1] positive negative neutral
Levels: negative < neutral < positive
```

The elements of a factor can also be modified using, for example an assignment. When we try to assign a value that is not within the allowed levels, an NA value will be inserted instead. To prevent this problem, the new level needs to be added.

```
> f[2] <- "positive"
> f
[1] positive positive negative neutral negative
Levels: negative < neutral < positive
> f[2] <- "strongly positive"
Warning message:
In '[<-.factor'('*tmp*', 2, value = "strongly positive") :
    invalid factor level, NA generated
> f
[1] positive <NA> negative neutral negative
> levels(f) <- c(levels(f), "strongly positive")
```

```
> f[2] <- "strongly positive"
> f
[1] positive strongly positive negative neutral negative
Levels: negative < neutral < positive < strongly positive
```

2.13 Data frames

In many situations, it is necessary to store records of data where individual elements of these records have different data types. In a scientific context, various experiments typically lead to a set of individual observations, each of which is described by the same set of measurements. This situation is also typical in relational databases and spreadsheets that both store data in a tabular format. Everyrows represents one record and the columns contain their properties. The values in every column are of the same type and all rows have the same number of data elements.

Data frames are used to store such data in R. To create a data frame, the values to be contained in the columns are provided as parameters. Each column must have the same number of elements or else, as many as can be recycled by an exact number of times (for example, if one column has 3 values, the other 4, and the last 12, everything is fine because the first column is used four times, the second three times, and the last one once).

The columns usually have names. When they are not provided, R creates some column names on its own. In order to enable duplicate column names, the check.names parameter needs to be set to FALSE. Alternatively, names for the rows can be provided in the parameter row.names or taken from any of the parameters that has names. When names are not provided, the rows are numbered.

```
> data.frame(c("a", "b"), c(1, 2))
    c..a...b.. c.1..2.
1 a 1
2 b 2
> data.frame(column1 = c("a", "b"), column2 = c(1, 2))
    column1 column2
1 a 1
2 b 2
> names <- c('John', 'Paul', 'Peter')
```

```
> ages <- c(20, 50, 25)
> children <- c(TRUE, FALSE, TRUE)
> df <- data.frame(names, ages, children)
> df
    names ages children
1    John 20 TRUE
2    Paul 50 FALSE
3   Peter 25 TRUE
> df <- data.frame(names, ages, children,
+ row.names=c("A", "B", "C"))
> df
    names ages children
A    John 20 TRUE
B    Paul 50 FALSE
C   Peter 25 TRUE
```

When a character vector is used as a part of a data frame the values are automatically converted to factors. To prevent this, the stringsAsFactors parameter needs to be set to FALSE.

```
> d <- data.frame(a=c("a", "b"))
> class(d$a)
[1] "factor"
> d <- data.frame(a=c("a", "b"), stringsAsFactors=FALSE)
> class(d$a)
[1] "character"
```

Data frames are implemented as lists where the columns are list elements. Thus, the length() function for a data frame returns the number of columns.

To extract individual columns from a data frame, we can use the same principles as when accessing lists. A notation using the number or name of a column within square brackets is possible.

```
> df [1]
    names
A John
B Paul
```

```
C Peter
> df["names"]
    names
A John
B Paul
C Peter
> df[c("ages","names")]
    ages names
A 20 John
B 50 Paul
C 25 Peter
```

The returned values are again lists. To access the content, a double square bracket or dollar notation can be used.

```
> df[["ages"]]
[1] 20 50 25
> df$ages
[1] 20 50 25
> class(df["ages"])
[1] "data.frame"
> class(df[["ages"]])
[1] "numeric"
> class(df$ages)
[1] "numeric"
```

More frequently, a pair of vectors containing indices (positive or negative integers, names, or logical values) is used to specify which rows and columns to extract. The first list refers to the extracted rows, the second vector specifies the columns.

```
> df[1:2, c("names", "children")]
    names children
A John TRUE
B Paul FALSE
> df[, c("names", "children")]
    names children
```

```
A John TRUE
B Paul FALSE
C Peter TRUE
> df[1:2,]
    names ages children
A John 20 TRUE
B Paul 50 FALSE
```

The returned value is, again, a data frame because severals rows consisting of columns of different data types are generally obtained. Only in the case that one column is returned, is the value is vector.

```
> class(df[1:2,])
[1] "data.frame"
> class(df[, 2])
[1] "numeric"
```

To add a new column to a data frame or to replace the values of an existing column, a vector of values can be easily assigned to an appropriate index (using a number or name). An alternative is the cbind() function which combines two data frames so the resulting data frame contains the columns from both data
frames (duplicate columns are not checked). The following commands have the same result.

```
> df[4] <- c("New York", "Boston", "Chicago")
> df[[4]] <- c("New York", "Boston", "Chicago")
> df["city"] <- c("New York", "Boston", "Chicago")
> df[["city"]] <- c("New York", "Boston", "Chicago")
> df$city <- c("New York", "Boston", "Chicago")
> df <- cbind(df,
+ data.frame(city=c("New York", "Boston", "Chicago"))
```

To delete a column, the NULL value is assigned to it.

```
> df["city"] <- NULL
```

When we need to add a row (or multiple rows) to a data frame, we actually have to merge two data frames. Both data frames must have the same column names (the order does not have to be the same).

```
> df <- rbind(df,
+             data.frame(ages=60,
+                        names="George",
+                        children=TRUE,
+                        city="Los Angeles",
+                        row.names="D"))
> df

    names   ages  children         city
A    John     20      TRUE     New York
B    Paul     50     FALSE       Boston
C   Peter     25      TRUE      Chicago
D  George     60      TRUE  Los Angeles
```

2.14 Function Useful in Machine Learning

Besides the functions that have been mentioned in relation with objects, their classes, data types, basic operations with numbers, strings, vectors, matrices, factors, etc., R provides many built-in functions which can be useful in a text mining process.

Let there be a matrix representing the frequencies of words in documents from three classes. To make the example simple, only nine different words are considered to be important. There are ten documents from classes labeled with numbers 1,2 , and 3 .

```
> # creating a data frame representing the distribution
> # of words in documents from three classes
> d <- data.frame(matrix(
+ c(3,0,0,0,1,0,0,0,1,1,
```

```
+ 0,1,1,1,0,0,1,0,0,1,
+ 0,0,1,0,1,0,0,1,0,1,
+ 0,0,1,0,1,0,0,1,0,1,
+ 1,0,0,2,1,1,0,0,0,2,
+ 0,0,0,1,0,1,0,0,1,2,
+ 0,0,0,0,1,2,1,0,0,2,
+ 1,0,0,0,0,0,3,0,2,3,
+ 0,0,0,1,0,0,1,2,1,3,
+ 0,0,2,0,1,0,2,0,1,3),
+ ncol=10,byrow=TRUE))
```

```
> colnames(d) <- c("football", "hockey", "run",
+ "president", "country", "Germany",
+ "software", "device", "communication",
+ "class")
```

To obtain some statistical information about the frequency of word Germany in documents (maximum, minimum, mean, etc.), we can call the summary() function.

```
> summary(d["Germany"])
    Germany
Min. :0.00
1st Qu.:0.00
Median :0.00
Mean :0.40
3rd Qu.:0.75
Max. :2.00
```

To see how frequent are words in documents on average, means of individual columns can be calculated using the colMeans() function.

```
> # obtaining average frequencies of all words
> colMeans(d[, -10])
    football hockey run president
         0.5    0.1 0.5       0.5
        country Germany software device
           0.5     0.4      0.8    0.3
communication
          0.6
```

Information about the typical document length and the variability of this property can be calculated using the functions mean() (arithmetic mean), var() (variance), and sd() (standard deviation).

```
> # obtaining the average length of a document
> # (not considering class labels)
> mean(rowSums(d[, -10]))
[1] 4.4
> var(rowSums(d[, -10]))
[1] 1.377778
> sd(rowSums(d[, -10]))
[1] 1.173788
```

Word frequencies and the distribution of documents in classes can be easily visualized (for more details about graphics, see 2.17).

```
> # visualizing global frequencies of all words
barplot(sort(colSums(d[, -10]), decreasing=TRUE))
> # a pie chart relative numbers of documents in classes
> pie(prop.table(table(d["class"])),
+     labels=paste("class",
+                  names(prop.table(table(d["class"]))),
+                  ":",prop.table(table(d["class"]))))
```

To see how individual pieces of data are related to each other, the function, table() can be used to create a contingency table of the counts at each combination of factor levels. It requires one or more objects which can be

interpreted as factors, or a list (or data frame) whose components can be so interpreted. In our example, we can observe, how often the words football and president or word run and individual class labels co-occur.

```
> # how often words co-occur displayed in a contingency table
> table(lapply(d[, c("football","president")],
+ function (x) x>0))
    president
football FALSE TRUE
    FALSE 4 3
    TRUE 2 1
> # how often a word and class label co-occur displayed
> # in a contingency table
> table(d[,"class"], d[,"run"] > 0,
+ dnn=c("Class label", "Contains 'run'"))
        Contains 'run'
Class   label FALSE TRUE
            1     1    3
            2     3    0
            3     2    1
```

Sometimes, it might be useful to find whether there are duplicates in the data. Logical values denoting indices of duplicated items in vectors or data frames (data frame rows) are returned by the function duplicated(). The supplied data structure is scanned according to the value of the parameter fromLast (FALSE by default) from the first or last element. If a current value has been found before, its index is marked as TRUE. The logical values can be used to index a structure to get the duplicates. If the values are negated, only the unique values can be obtained.

```
> # getting indices of duplicated items
> duplicated(d)
    [1] FALSE FALSE FALSE TRUE FALSE FALSE FALSE FALSE
    [9] FALSE FALSE
> # removing the duplicates
> d <- d[!duplicated(d), ]
```

The function cut() is useful when the values of a numeric vector need to be divided into intervals and coded according to the interval in which they fall. The leftmost interval has level one, the next leftmost level two and so on. The values can be assigned to the given number of intervals or interval boundaries which are supplied (breaks parameter). If labels are provided, they are used to code the intervals instead of numbers. In our example, class label 1 will be converted to sport, the remaining labels to other.

```
> cut(d$class,
+ breaks=c(0, 1, 3),
+ labels=c("sport", "other"))
[1] sport sport sport other other other other other other
Levels: sport other
> # replacing numeric class labels with strings
> d$class = cut(d$class,
+               breaks=c(0, 1, 3),
+               labels=c("sport", "other"))
```

For some tasks, working only with a part of a data set might be needed. An example is bagging (Bootstrap aggregating), a method for creating an ensemble of classifiers trained on samples of data [32]. Another situation is when there are so many data instances available so that they cannot fit into the memory to be processed.

Sampling can be achieved using function sample(). It requires a vector of elements from which some will be chosen, or an integer (a sample will then be created from values 1 : x). The number of elements is given by the size parameter. The elements can appear once (sampling without replacement, parameter replace is set to FALSE) or more than once in the selection (sampling with replacement, parameter replace is set to TRUE).

```
> # creating a sample containing 50% instances from
> # the entire data set
> d[sample(nrow(d), size=nrow(d)*0.5, replace=FALSE),]
  football hockey run president country Germany
3        0      0   1         0       1       0
9        0      0   0         1       0       0
6        0      0   0         1       0       1
1        3      0   0         0       1       0
  software device communication class
3        0      1             0 sport
9        1      2             1 other
6        0      0             1 other
1        0      0             1 sport
```

The function sample() can be easily used to randomize rows in a data frame. Shuffling, for example, training data after a few epochs of training can improve the performance of a learning system [51].

```
> # shuffling rows of d
> d <- d[sample(nrow(d)),]
```

Often, a function (transformation) needs to be applied on elements of a data structure (raising numbers in a vectors to a power of two, calculating sums of rows in a matrix, etc.). Instead of using loops to iterate over the structures, function apply() and other similar functions are a good choice.

The function apply() takes an array when an object of another type is supplied; R tries to coerce it to a matrix using as .matrix() and applies a function to its elements. The elements are defined by the second parameter (MARGIN), which, in the case of matrices, is 1 for rows, 2 for columns, c(1, 2) for both rows and columns (i.e., for each matrix element), or a list of dimension names when the dimensions have names. The function that is supplied as the third parameter (FUN) can be one of existing functions or a user defined one. Function apply() returns a vector or array or list of values according to the type of data returned by the supplied function.

```
> # counting the numbers of words in every document
> # (applying function apply() to rows)
> apply(d[, -10], 1, sum)
10 5 1 6 8 3 9 7 2
 6 5 5 3 6 3 5 4 4
\end{tabular}
```

```
> # counting the frequencies of words in all documents
> # (applying function apply() to columns)
> apply(d[, -10], 2, sum)
       football    hockey             run
              5         1               4
      president   country         Germany
              5         5               4
       software    device   communication
              8         3               6
```

```
> # a user defined function is used to threshold frequencies
> # of words in documents (if a frequency is greater than 1
> # it is set to 2)
> apply(d[, -10], c(1, 2), function (x) if (x > 1) 2 else x)
   football hockey run president country Germany
10         0      0   2         0       1       0
5          1      0   0         2       1       1
1          2      0   0         0       1       0
6          0      0   0         1       0       1
8          1      0   0         0       0       0
3          0      0   1         0       1       0
9          0      0   0         0       0       0
7          0      0   0         0       1       2
2          0      1   1         1       0       0

  software device communication
10      2       0             1
5       0       0             0
```

```
1       0       0               1
6       0       0               1
8       2       0               2
3       0       1               0
9       1       2               1
7       1       0               0
2       1       0               0
```

Function lapply() is able to apply a supplied function to all elements of the passed object. In comparison to apply(), it does not require the MARGIN parameter. It works with vectors or lists (other objects are coerced to lists) and returns a list of the same length as that of the object. The same functionality is provided by the sapply() function that returns a vector.

Function which() returns the position where a logical object has TRUE values. If the logical object is an array, indices can be returned instead of scalar positions (parameter arr. ind must be set to TRUE).

```
> # obtaining positions of documents from class 2
> which(d$class == "sport")
[1] 3 6 9
```

2.15 Flow Control Structure

R provides a variety of control flow structures that are typical in many other programming languages. The flow control statements are all reserved words. A part of these statements is sometimes a condition based on which the code is executed. A condition is a logical vector of length one. If a longer vector is provided, only the first element will be processed.

2.15.1 Conditional Statement

The conditional statement is used when we need to decide whether to execute a statement or not or in the case where one of two possible statements is to be executed. The statement has a common form:

if(*condition*) *expression*

or, alternatively:

if (*condition*) *expression 1* else *expression 2*

When the condition is evaluated as TRUE, expression 1 is evaluated. If the condition is evaluated as FALSE, nothing happens. If the else statement exists, expression 2 will be evaluated in this case.

```
> x <- 2
> y <- 1
> if (y != 0) x/y else "Cannot divide by 0"
[1] 2
```

If an expression to be evaluated consists of more than one statement, the statements must be surrounded by curly braces. It is important to note that the else keyword must be on the same line as the closing bracket of the if branch in the console mode.

```
> day <- "Saturday"
> if (day == "Saturday" | day == "Sunday") {
+ print("It is the weekend")
+ print("It is good")
+ } else
+ {
+ print("It is a work day")
+ }
[1] "It is the weekend"
[1] "It is good"
```

When there are more than two alternative statements to be executed, we can simply use a nested conditional statement.

```
if (conditionl) {
    # condition 1 is TRUE
    if (condition 2) {
        # condition 2 is TRUE
        ...
    } else {
        # condition 2 is FALSE
```

```
        ...
    }
} else {
    # condition 1 is FALSE
    ...
}
```

But what if we have many more alternatives to choose from? A structure with many nested conditional statements will became confusing and difficult to maintain. The switch statement can handle these situations effectively. It has the following syntax:

`switch(expression, ...),`

where . . . represents a list of options. The switch statement accepts a string or number as the first argument (if the argument is not a string it is coerced to integer). When a number is the first argument the corresponding expression from the list of options will be evaluated and returned:

```
> day <- 2
> switch(day,"Mon","Tue","Wed","Thu","Fri","Sat","Sun")
[1] "Tue"
```

If the first argument is a string, an exact match with the names of the elements in the option list is searched. Then, the found element is evaluated and returned:

```
> day <- "Tue"
> switch(day,
+        Mon = "Monday",
+        Tue="Tuesday",
+        Wed="Wednedsay",
+        Thu="Thursday",
+        Fri="Friday",
+        Sat="Saturday",
+        Sun="Sunday")
[1] "Tuesday"
```

The `ifelse` function can be used for vectorized decisions. The returned value has the same shape as the first parameter. Its values are filled with values from the second or third parameter depending on whether the values of the first parameter are evaluated as TRUE or FALSE.

```
> x <- 1:5
> ifelse(x>3, 'greater than 3', 'less or equal to 3')
[1] "less or equal to 3" "less or equal to 3"
[3] "less or equal to 3" "greater than 3"
[5] "greater than 3"
> letters <- c("a", "b", "c", "d", "e")
> digits <- c(1, 2, 3, 4, 5)
> ifelse(c(TRUE, FALSE, TRUE, TRUE, FALSE), letters, digits)
[1] "a" "2" "c" "d" "5"
```

2.15.2 Loops

R provides three basic types of loops. The simplest loop, repeat, only repeatedly evaluates the given expression (infinite loop). The break keyword is used to terminate the loop.

```
> i <- 1
> repeat {print(i); i <- i+1; if (i>5) break}
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
```

The while loop implements a loop with the condition at the beginning. It repeats an expression as long as the loop condition is true.

```
> i <- 5
> while (i>0) {print(i); i <- i-1}
[1] 5
```

```
[1] 4
[1] 3
[1] 2
[1] 1
```

The break command can be used inside a while loop too. It terminates the loop here as well. In addition, the next statement skips the remaining statements in the loop body and starts a new loop iteration (evaluating the condition).

The for loop is used to iterate through each element of a vector or list. These elements are assigned to a variable, the name of which is a part of the for statement.

```
> a <- c(1, 2, 3)
> b <- c("a", "b", "c")
> for (i in a) print(i)
[1] 1
[1] 2
[1] 3
> for (i in 1:length(a)) print(a[i])
[1] 1
[1] 2
[1] 3
> for (i in c(a,b)) print(i)
[1] "1"
[1] "2"
[1] "3"
[1] "a"
[1] "b"
[1] "c"
```

The variable is, however, not an alias for the iterated values:

```
> a <- c(1, 2, 3)
> for (i in a) i <- i+1
> a
[1] 123
```

2.16 Packages

R offers many readily available functions. Sometimes, however, functions for other different tasks are needed too. The functionality other than that provided by the R core team is accessible through so called packages.

A package is set of functions, data, and help files that have been bundled together in the library subdirectory (the directory containing a package is called the library).

A package typically contains functions that are related to a specific task. They were written by the R community to solve problems in various disciplines. A core set of packages is already included in the installation of R. Other packages are stored and available in the official R repository known as *CRAN* (Comprehensive R Archive Network). CRAN is a collection of sites which carry identical material about R , including the R distribution, extensions, documentation, and binaries. The CRAN master site (https://CRAN.R-project.org/) is maintained at Wirtschaftsuniversität Wien, Austria and a complete list of mirrors can be found at `http://CRAN.R-project.org/mirrors.html`. CRAN includes approximately 13,000 additional packages and a list of the same can be found at https://cran.rproject.org/web/packages/.

To see the list of packages loaded by default, use the command getOption ("defaultPackages"). The result omits the base package, which is loaded every time.

In order to view the list of all packages installed by the user, the installed.packages() function can be called. Alternatively, the Packages pane in RStudio shows a table with this information.

To get the list of the packages currently loaded into R , we can use the function .packages().

2.16.1 Installing Packages

To be able to use a package, it has to be downloaded and installed on a computer. Use the install.packages() function to install a CRAN package in R. The parameter is a name of the package to be installed. This function downloads the package from a repository and installs it locally. In order to install two or more packages at the same time, a list of package names can be passed as the parameter.

In RStudio, it is possible to use the bottom-right pane Packages and the Install button. Be sure the Install dependencies option is checked.

An example of running a code where the necessary library is not installed

can be found in Section 5.6.1.

Sometimes, packages are updated by their creators. To check which packages have been modified, we can use the old.packages() function. If a newer version is preferable for us, we can call update.packages(). This will update all packages interactively. If the argument ask is set to TRUE (default value), packages with a newer version are reported and for each one we can specify whether we want to update it. If we only want to update a single package, the best way to do it is to use install.packages() again. In RStudio, the Update button in the Packages pane can perform this task too.

To get some basic information about a package, use the library() function with the help parameter set to the name of the package, e.g., library(help="-tm").

2.16.2 Loading Packages

To load a package in R , we call the library() function where the parameter is the name of a package. We may also use the function require(). It is a similar function which accepts a slightly different set of arguments.

Functions from a package are available after loading the package. If more functions with the same name are in more than one package, the one that has been loaded last will be called. To call another one, a function name can be preceded by a package name and :: .

To unload a package, we can call the detach() function.

2.17 Graphics

R includes three graphical systems. One of them is represented by the standard graphics package, the others use the lattice package and the ggplot2 package. R includes tools for drawing most common types of charts like scatterplots, line charts, bar charts, pie charts, histograms, etc. as well as any complicated graphics. To create a graphical output, a series of functions are called. They create complete plots (high-level functions) or add some elements to existing plots (low-level functions). This approach follows the so called painters model where the graphics are created in steps and newer objects overlap the previous output [204]. The graphical output is printed on the screen (in one of the panes of RStudio) or can be saved in a wide variety of formats.

In this book, we will focus on the graphics package which includes the capabilities for some basic graphics while providing a wide variety of func-

tions for plotting data. We present only some basics that you might need to draw and customize simple graphs. To see some basic information about the graphics package and its functions, call library(help="graphics").

The most important function for creating graphics is plot(). It accepts objects as parameters and tries to plot them. In the simplest case, the arguments are values on the x and y axes. The graph is created according to the supplied parameters. To display all the argument names and corresponding default values of the plot() function, run the args(graphics::plot.default) statement.

To demonstrate some basic capabilities of the plot() function, we will use fictive data - numbers of newspaper articles from different categories (classes) published in five days of a week.

```
> articles <- matrix(c(3,10,6,8,15,9,2,9,4,6,8,3,3,7,1),
+                    nrow = 5,ncol = 3,byrow = TRUE)
> dimnames(articles) = list(
+     c("Mon", "Tue", "Wed", "Thu", "Fri"), # row names
+     c("Economy", "Politics", "Science")) # column names
> articles

  Economy Politics Science
Mon     3       10       6
Tue     8       15       9
Wed     2        9       4
Thu     6        8       3
Fri     3        7       1
```

To create a simple plot with the number of released articles in the Economy category on the y axis (see Figure 2.8), we can call plot (articles [, 1]) or plot(articles[, "Economy"]).

We can change the way the values are displayed by providing a value for the type parameter, e.g., p for points, 1 for lines, b for both, o for both overplotted, h for histogram, or s for stair steps. The title is defined by the main argument or can be added by calling the title() function.

```
> plot(articles[,"Economy"], type = 'o')
```

```
> title(main="Articles numbers")
```

In the following example, we first display the plot with the number of articles from the Economy category. No axes are displayed (the axes parameter), they will be added later, the line color will be changed to blue (the col parameter), the y-axis will be in the range 0 to the highest value in articles matrix (the ylim parameter), and the axes will not be annotated (the ann parameter). Subsequently, we will add the main graph title, the x and y axes (at defines the points at which tick-marks are to be drawn, the labels parameter provides the labels), descriptions of the axes in dark grey color, and a box around the graph, see Figure 2.9).

```
> articles_range=range(articles)
> plot(articles[, "Economy"], type="o", col="blue",
+      axes=FALSE, ann=FALSE, ylim=articles_range)
> title(main="Numbers of articles")
> axis(1, at=1:5, labels=rownames(articles))
> axis(2, at=2*0:(articles_range[2]/2))
> box()
> title(xlab="Days", col.lab="darkgrey")
> title(ylab="Number of released articles",
+       col.lab="darkgrey")
```

We can add another line for the articles from the Politics category. It will be red in color, the line will be dashed, and the points will be square in shape. To clarify what is on the graph, a legend will be added. It will be placed on a specified position in the graph (coordinates [4, 15]), the text will be slightly smaller (the cex parameter), and the same line colors (the col parameter), point types (the pch parameter), and line types (the lty parameter) used by the actual plots will be used, see Figure 2.10).

```
> lines(articles[,"Politics"], type="o", pch=22,
+       lty=2, col="red")
> legend(4, 15, c("Economy","Politics"), cex=0.9,
+        col=c("blue", "red"), pch=21:22, lty=1:2)
```

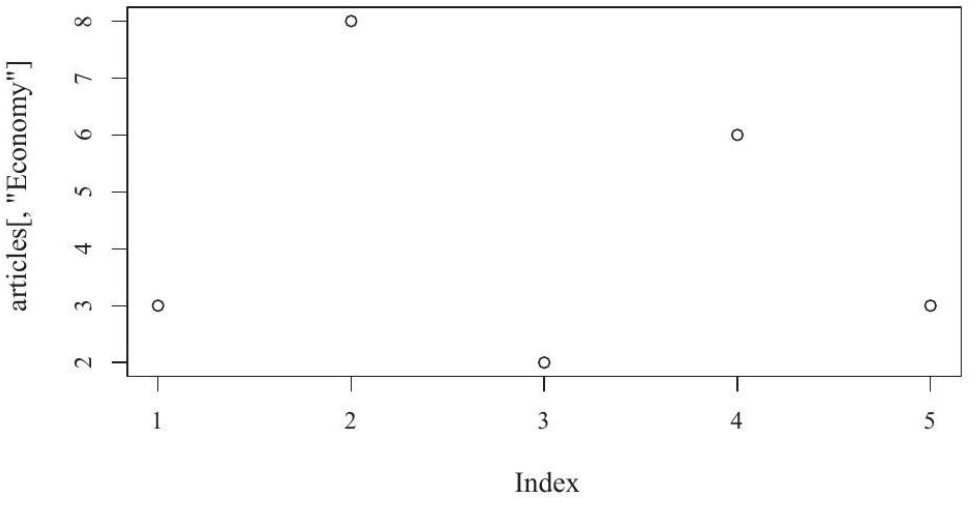


Figure 2.8: The simplest X-Y plot.

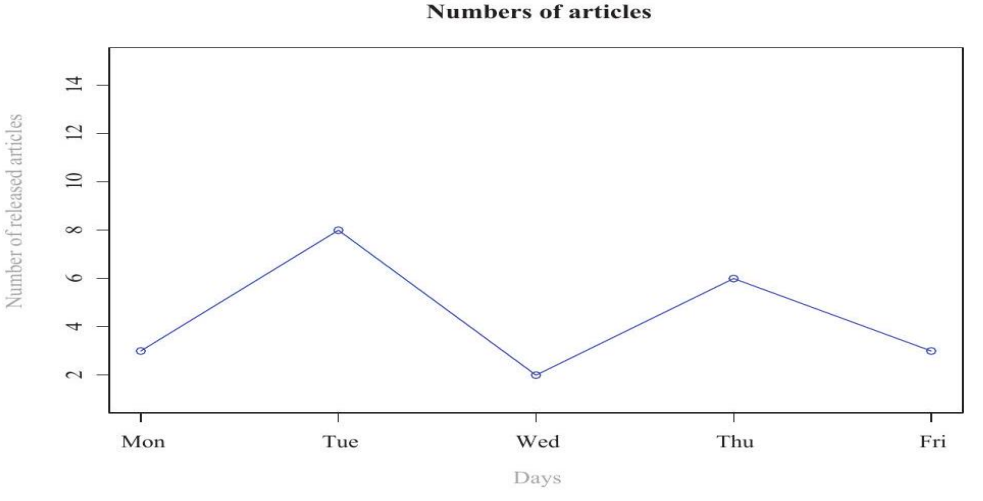


Figure 2.9: A customized X-Y plot.

The plot() function is a good choice if you want to plot one or two columns in one chart. When more data segments are to be displayed in a plot, another function, matplot(), can be used instead (see Figure 2.11).

```
> matplot(articles, type ="b", pch=1:3, col=1:3,
```

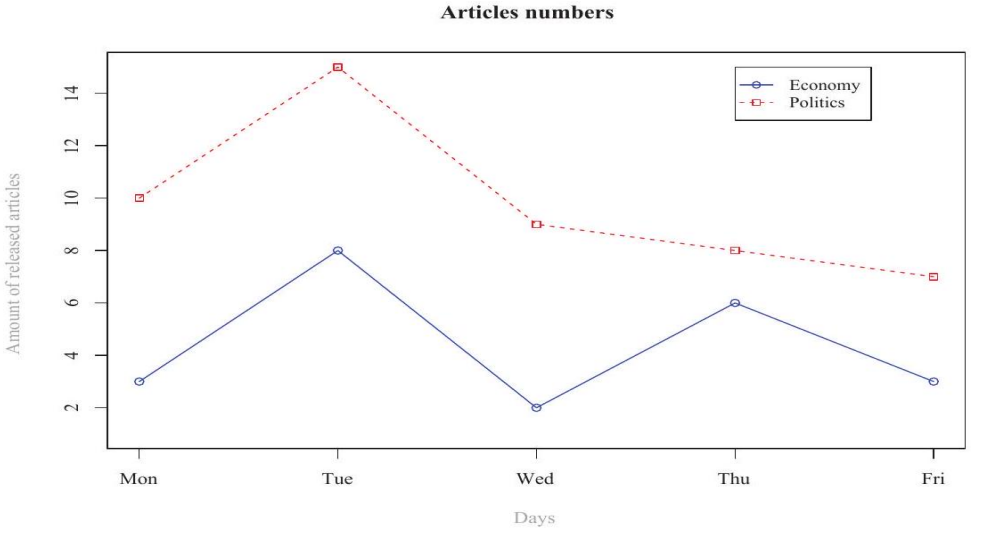


Figure 2.10: A customized X-Y plot with two data segments.

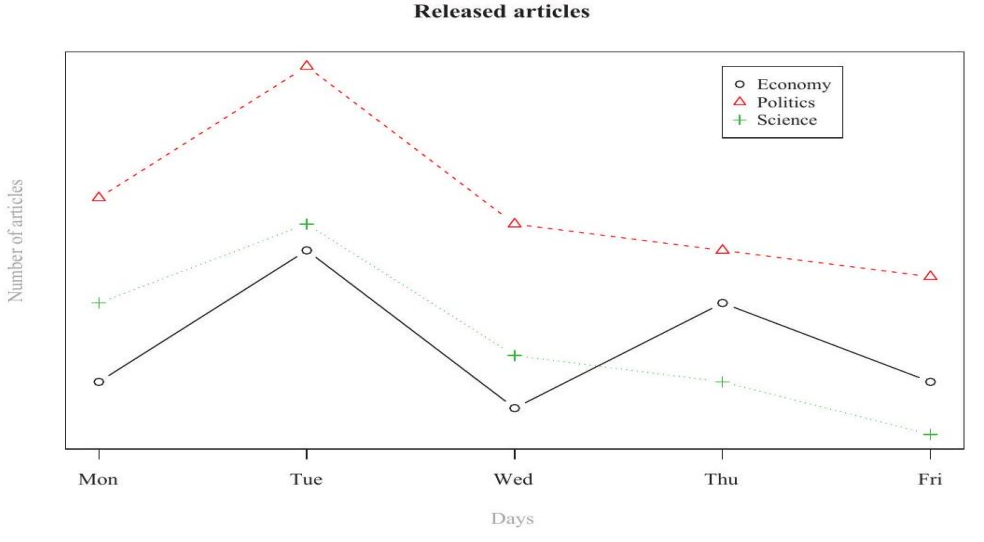


Figure 2.11: A customized X-Y plot with three data segments.

```
+         axes=FALSE, ann=FALSE)
> title(main="Released articles")
> axis(1, at=1:5, lab=rownames(articles))
> axis(2, las=1, at=2*0:released_articles[2])
```

```
> box()
> title(xlab="Days", col.lab='darkgrey')
> title(ylab="Number of articles", col.lab='darkgrey')
> legend(4, 15, names(articles[1,]), cex=0.9,
+        col=1:3, pch=1:3)
```

A pie chart is a good option to show some proportions of a whole, for example, when we want to show how are the newspaper articles in the Economy category are distributed in a week. This type of graph is created with the pie() function. The first parameter is a vector of non-negative numbers which are displayed as areas of pie slices. Other parameters include labels to be assigned to the slices, colors of the slices, or a title for the chart.

```
> colors <- c("green", "blue", "yellow", "orange", "red")
> pie(articles[, 1], main=names(articles[1, ])[1],
+     labels=names(articles[, 1]), col=colors)
```

Slices with the specified colors, labeled with days are displayed in the chart created. In the following example, we will add also the absolute numbers of released articles together with the percentages rounded to one decimal place, and a legend, see Figure 2.12).

```
> percent <- round(articles[, 1]/sum(articles[, 1]) * 100, 1)
> percent <- paste(percent, "%", sep="")
> pie(articles[, 1],



+ main=names(articles[1, ])[1],
+ col=colors,
+ labels=paste(paste(names(articles[, 1]),
+ articles[, 1],
+ sep=": "),
+ percent, sep="="))
> legend("topright", names(articles[, 1]),
+ cex=0.9, fill=colors)
```

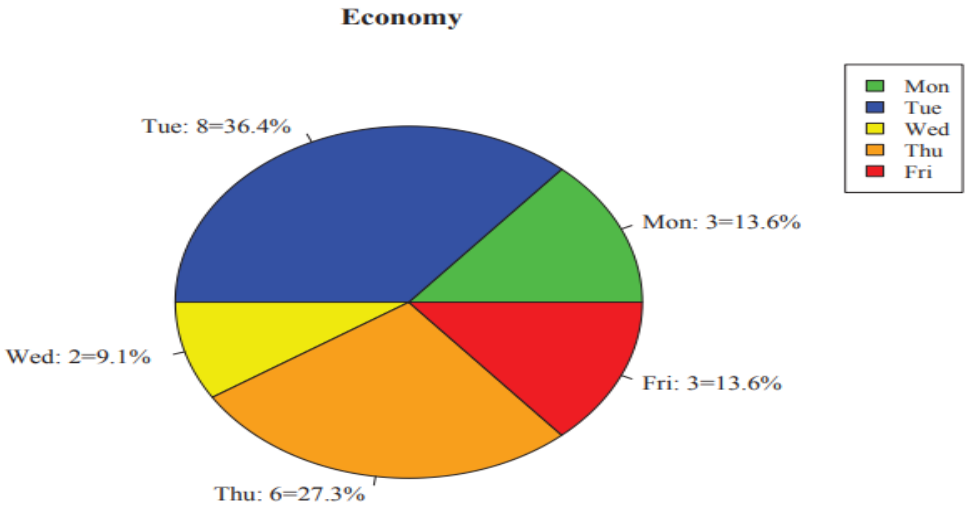


Figure 2.12: Pie chart with labels and a legend.

Figure 2.12: Pie chart with labels and a legend.

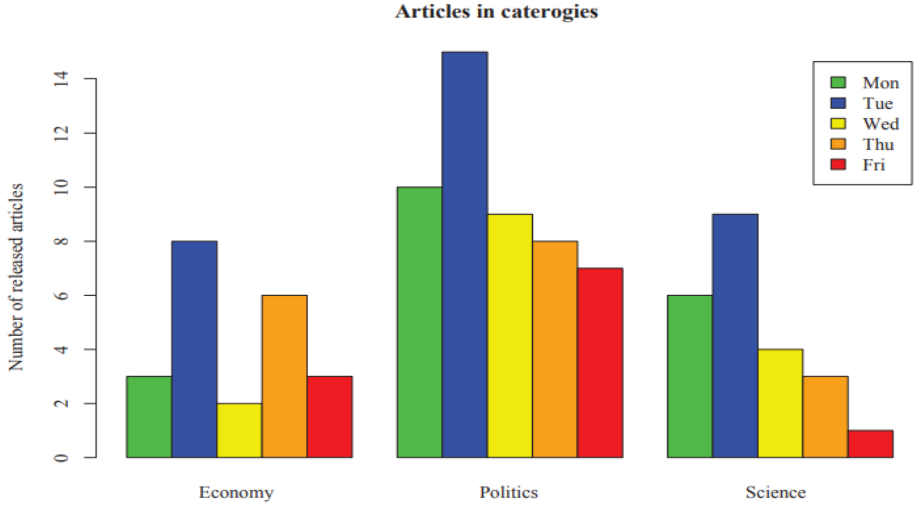


Figure 2.13: Bar chart.

Figure 2.13: Bar chart.

A simple bar chart can be created using the barplot() function. If the

first(or the height) parameter is a vector, a sequence of bars representing the numeric values of the vector will be created.

```
> barplot(articles[, 1])
```

If the height parameter is a matrix, the bars correspond to the columns of the matrix. The values in the columns are either stacked in one bar or represented by a sequence of other bars (depending on the logical beside parameter). In our example, the x -axis will represent the newspaper categories and the sub-bars the days of the week. The $t()$ function can transpose the matrix so we can change the perspective on the data (i.e., days on the x -axis and categories as sub-bars).

```
> barplot(articles)
> barplot(articles, beside=TRUE)
> barplot(t(articles))
```

We can play with many parameters, for instance, adding descriptions of axes, changing colors, or adding a legend.

```
> barplot(articles, main='Articles in caterogies',
+ ylab='Number of released articles',
+ beside=TRUE, col=colors, legend=TRUE)
```

We can customize many features of the graphical output through setting graphical options using the par() function. This function, called without parameters, returns a list of the values of current parameters. To change the parameters, a list of tag=value pairs are submitted. The parameters include, for example, justification of texts, margins, font, foreground or background colors, and many others.



Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

Chapter 3

Structured Text Representation

3.1 Introduction

Structured data is usually organized in databases, particularly relational databases. The high level organization, typically represented by tables, enables easy and efficient processing, for instance, searching or filtering. Text data is usually considered to be unstructured. A newspaper article, e-mail, SMS, or a recipe in a cookbook definitely does not look like a table. However, because the texts are normally generated using the grammar of a natural language and rules of a certain linguistic style, they have some kind of structure. For example, in a scientific paper, parts like a title, authors, abstract, keywords, introduction, etc. are expected or required. Every part consists of one or more words, symbols, expressions, sentences, organized into paragraphs, sections, or subsections. Every sentence contains words from several classes (e.g., nouns, verbs, adjectives) that are combined in the form of verb, noun, or prepositional phrases so they can bear some meaningful information. The words belong to a dictionary of the given natural language and their combinations are driven by the rules of the language. The rules are known as the grammar. There exist many rules (types of a sentence or expression) in a language and it is possible to create infinite texts written in a specific language. However, the structure is very complicated and it is not possible to describe all texts from one domain for example, the newspaper articles about the Winter Olympic Games in 2018 with one structure (template). An example of a structure would be all visitors of the Grandezza hotel in Brno, where every visitor has a name, surname, data of birth, address, etc. It is obvious that the second example data (hotel visitors) distinguishes some entities where all entities from the same class have the same attributes (properties). This is not true for the newspaper article text.

There exists another form of data between structured and unstructured data. It is the data that has some structure but this structure does not conform to what is normally expected from structured data. It means that this structure cannot be directly converted to a structure of a relational database (tables). This data is known as semi-unstructured. Textual documents, especially those on the web, are a typical representative of this. In the semi-structured form, the structure is usually expressed by using some tags or other marks which identify and separate pieces of data and thus, enable the creation of some data records and their relationships, often hierarchical (a tree of elements). The schema of semi-structured data does not have to be fixed in advance, which provides a high flexibility; hence, all instances of the same entity do not have to share the same attributes. Redundancy or non atomic values are also allowed [173, 219]. Typical representatives of semi-structured formats are XML or JSON.

It would be very complicated to process texts without some kind of simplification, i.e., giving them a better structure so they can be more easily understood by machines. In fact, most machine learning algorithms require the data to be processed in a highly structured format. A structured format, as we already mentioned, is usually something that can be represented by a table. In the rows, there are some objects to be processed. In the case of text documents, those objects might be e-mails to be classified as spam (unsolicited, junk e-mails) and ham (legitimate, relevant e-mails). In the columns of the table, there are features (properties, characteristics) of the objects and their specific values. Every object is thus represented by a feature vector that characterizes it.

The question is, how to derive the features from unstructured texts. In the case of e-mails (in the spam filtering problem), those features might be a result of tests performed by the SpamAssassin filter [89]. These tests include examinations whether an e-mail contains images, an HTML code, empty lines, words like cialis, viagra, or Your Bills, whether the sender is known or not, is in different blocklists etc. We might include other properties, such as the length of a document in words, the date of publication, author, number of figures and tables, language, and so on. In a majority of problems, however, the content of the documents needs to be taken into consideration. It means that the words, their combinations and meaning are of the highest importance. The document features are thus often derived from the words in the document.

A widely used format in the text mining domain is the vector space model proposed by Salton and McGill [236]. Every document is represented by a vector where the individual dimensions correspond to the features (terms) and

the values are the weights (importance) of the features. All vectors then form so called a document-term matrix where the rows represent the documents and the columns correspond to the terms in the documents. The features often correspond to the
words contained in the documents. In other words, the particular property of a document related to a specific word is determined by the occurrence of the word in the document.

Let's take the following group of documents (fictive news headlines):

- *The Vegas Golden Knights ice hockey team achieved the milestone of most home wins.*
- *Germany won 1-0 over Argentina in the FIFA Wold Cup finals.*
- *Someone has won tonight's USD 10 million jackpot in the National Lottery.*
- *A man has won a USD 5 million jackpot on a Las Vegas slot machine.*

From the words and expressions used in the sentences, one can conclude that the first two documents are about sports and the remaining two talk about gambling. How could we arrive at such a conclusion? In the first two documents, there are words like ice, hockey, and FIFA that are almost exclusively related to the sports domain. In the last two documents, words like jackpot or lottery are probably the most notable. We might therefore say that if a document contains the word hockey it is most likely related to sports. In other words, the document has the property of having the word hockey included in the text.

For the purpose of assigning a document to a particular category, the position of the word is sometimes not important. We might change the first sentence to The ice hockey team of Vegas Golden Knights achieved the milestone of most home wins. The word hockey moved to the third position in the sentence from the sixth position and the context (the surroundings, a few words before and after the word) also changed. However, the meaning remains the same and it is still a sentence related to sports.

It seems that treating the content of a document as a set of words appearing in it is sufficient in some situations, such as assigning document to a category. In such a set, any relations with the remaining elements are not relevant. These relations in a sentence represent the context of the words - which other words are in the same sentence, which words are before and which are after, at what distance they are, etc.

Not considering these relationships, however, definitely causes some information to be lost. Let us look at the word Vegas. Without considering its con-

```
IF document contains ‘‘hockey’’  THEN document category = sport

IF document contains ‘‘lottery’’ THEN document category = gambling

IF document contains ‘‘Vegas’’ AND
   document contains ‘‘Knights’’ THEN document category = sport

IF document contains ‘‘machine’’ AND
   document contains ‘‘slot’’    THEN document category = gambling

...
```

Figure 3.1: Examples of rules used for categorization of documents into two categories (sport and gambling).

text, this word is related to both topics - sports and gambling. The relations to some topics are not clear also for some other words, like ice, Golden, World, or machine. Knowing their context, the situation might change significantly. If we have expressions like ice hockey, Vegas Golden Knights, World Cup, or slot machine, their meaning is much clearer than in the case of handling each word separately. The solution might be to not consider only isolated words but also sequences of two, three, or even more words (known as n-grams) or treating some word sequences as named entities (e.g., Vegas Golden Knights is a name for an ice hockey team, Las Vegas is a name of a place).

The algorithms that are used to analyze texts often work properly with a combination of documents rather than just one document. The rules used to categorize the documents can take the form shown in Figure 3.1.

This means, that even when the words are treated independently, a state when their context is somehow taken into consideration can be achieved. For example, the Naïve Bayes or decision tree learners handle the features independently. After training, a representation consisting of multiple attributes (the joint probability calculation and decision tree branches) is created (see Chapter 5). The independence of attributes is also sometimes necessary for making the problem feasible, as in the case of training the Naïve Bayes clas-

sifier. Imagine that we have 1,000 unique words ($n = 1000$) and we want to use them to create a document of 50 words ($r = 50$), where some of the words might appear more than once. Keeping in consideration the order of the words, there exist n^r possibilities (permutations), which is 10^{150}. When the order of words is not important, the number of possible documents (combinations) is $(r+n-1)!/(r!(n-1)!$, which is about $1.097 * 10^{86}$. It is thus obvious that consideration of the word order significantly increases the complexity of the solved problems.

Documents are therefore often treated as sets of independent words or expressions. One word might appear multiple times in one document. The number of occurrences can also play an important role. The more times a word appears, the more important it probably is. Sets generally do not allow multiple occurrences of items. An exception is so called multisets or bags that allow multiple instances of their elements. In the text mining domain, the later term is commonly used and the bag-of-words model is a model commonly used in natural language processing. It become very popular because of its simplicity and the straightforward process of creation while providing satisfactory results [135].

3.2 The Bag-of-Worlds Model

In the bag-of-words model, the features that are derived from the processed documents are based on the words contained in them. Each document is thus represented by one bag (multiset). The words that appear in the document are present in this set. The words that are not present in the document are not in the set. The sets can be converted to vectors that are a more common data representation in machine learning. This approach is depicted in Figure 3.2. We can see that different documents are represented by sets of different sizes (containing different numbers of elements). The vectors created from the sets are also of different sizes. And what is more, we can see that the vector components (table headers) have different meanings. For example, the first component of the first vector represents the presence of the word good whereas the first component of the second vector represents the presence of the word bad. This is not a desired state when we want, for example, to compare these two documents. They have the same value of the first feature but the semantic meaning is opposite. Additionally, most of the machine learning algorithms require the vectors to have a fixed length.

These considerations lead us to the conclusion that, in the ultimate rep-

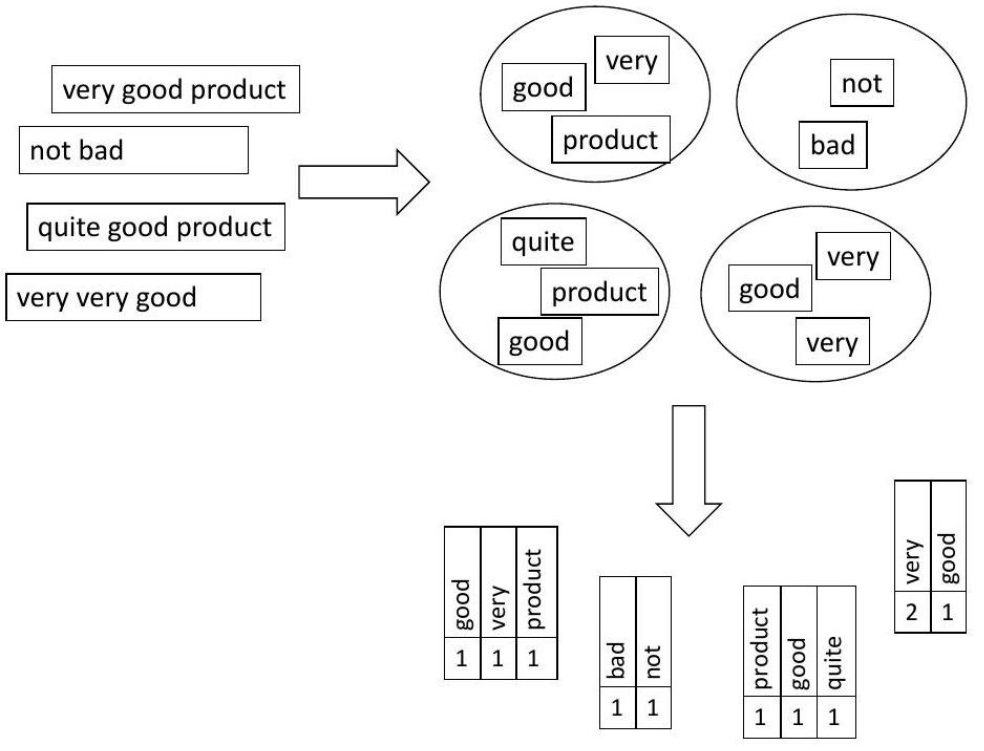


Figure 3.2: The principle of transforming documents (here, examples of very short product reviews) to bag-of-words.

resentation of individual documents to be processed, all documents must have the same number of features and we must be able to clearly identify each feature across the entire document collection. Thus, the attribute set for the collection must be able to accommodate all possible features and all documents will share the same features. Of course, some of the documents may not be characterized by some features (i.e., their value is 0), see Figure 3.2. Now, all documents in a collection are represented by vectors of the same type and can be arranged into a table known as the term-document matrix. Such a matrix has a highly structured format which is suitable for many machine learning algorithms.

very	good	not	bad	quite	product
1	1	0	0	0	1
0	0	1	1	0	0
0	1	0	0	1	1
2	1	0	0	0	0

Figure 3.3: The principle of transforming documents (here, examples of very short product reviews) to bag-of-words.

3.3 The Limitations of the Bag-of-Words Model

The application of the bag-of-words approach in creating the term-document matrix has a few disadvantages. First, there exist a large number of possible features which can be derived from texts. When considering words as the most natural features, even quite a small collection of documents can contain many unique words. In a data set consisting of 100 hotel accommodation reviews from booking.com where the average number of words in a review is about 25 , almost 1,000 unique words can be found. In 1,000 documents, we identified more than 3,500 unique words, a collection having 10,000 documents contained almost 13,000 unique words, and in 100,000 documents, about 45,000 distinct words could be distinguished [295]. Such a high number of features is significantly different from what is typical in classic data mining problems or in relational databases.

The high number of features is often a problem from the computational complexity perspective because processing a larger number of attributes requires more time. For some algorithms, for example a decision tree induction, the computational complexity grows quadratically with the number of features [144].

Another problem related to the high number of dimensions is known as the curse of dimensionality. Inductive machine learning algorithms try to find a generalization of the available data. In the case of supervised classification, the goal is to find a function that is able to assign every item to a correct category (class). Having a data set with n binary (or boolean) attributes, there exist 2^n different instances. When there are 100 binary attributes, there exist $1.27 \cdot 10^{30}$ instances. To learn a perfect function covering the entire instance space, we would need a huge number of these instances. This is,

however, often neither possible nor feasible. Because of that, we expect that the classifiers will work with some errors
and not so many learning instances are needed. With the use of the PAC-learning (probably approximately correct learning) paradigm, we can even quantify the relationship between these numbers [150].

The curse of dimensionality is a bigger problem for some algorithms that for others. For example, a k-nearest neighbors (k-NN) classifier uses all features to calculate a similarity between the items in a data set. Thus, the k-NN classifier is more sensitive to the high number of attributes than, for example, a decision tree or rule learners that work with only a subset of attributes when creating a generalization of the data [199].

There are two famous statistical laws applicable to natural languages - the Zipf's law and Heaps' law Zipf's law says that the frequency of a word (or another linguistic unit, like a word stem) is inversely proportional to its rank derived from the frequency of the word in the collection of texts [186]. This relationship can be represented by the following equation:

$$\mathit{freq}\,(w_i) = \frac{C}{i^a}, \tag{3.1}$$

where $\mathit{freq}\,(w_i)$ is the number of occurrences of the word w_i and i is the position of the word w_i in the list of all words ordered in descending order according to their frequency (in other words, i is the rank of the word w_i). The coefficients C and a can be derived from the available data having the word counts for every word appearing in the document collection. Ideally, $C = \mathit{freq}\,(w_1)$ and $a = 1$. In that case, the frequency of the second most frequent word will be one half of the frequency of the most frequent word, the frequency of the third most frequent word will be one third of the frequency of the most frequent word, etc. The frequency of the 10th most frequent word will be twice as much as the frequency of the 20th most frequent word and three times the frequency of the 30th most frequent word. Zipf's law was slightly modified by Mandelbrot [184] to better represent the law, especially for words with high and low ranks:

$$\mathit{freq}\,(w_i) = \frac{P}{(i+\rho)^B}, \tag{3.2}$$

where $\mathit{freq}\,(w_i)$ is the number of occurrences of the word w_i, i is the rank of the word w_i, and the coefficients P, ρ, and B are derived from the data.

Heaps' law [114] describes the relationship between the size of a data collection as expressed by the number of words and the number of unique

words. This relationship is demonstrated by the formula:

$$M = k \cdot N^b, \tag{3.3}$$

where M is the number of unique words, N is the total number of words, and k and b are coefficients derived from data. Manning, Prabhakar and Schütze [185] derived the values $k = 44$ and $b = 0.49$ for the Reuters-RCV1 data set. The typical values of k are between 30 and 100, b is close to 0.5 (the number
of unique words is thus proportional to the square root of the total number of words).

The coefficients that appeared in the formulae representing the laws of natural languages often depend on a specific language; for example, how inflective it is [98]. However, the laws can give us at least some initial insight to the data and its statistical characteristics.

For example, when considering the first 10 most frequent words, they should form about one third of all words in a document collection keeping in mind the ideal set that conforms to Zipf's law. Since the data is not ideal, this is not very exact. However, Manning, Prabhakar and Schütze [185] found that the 30 most common words account for about 30% of the tokens in Reuters-RCV1. Not using these words as features in a machine learning task will, on the one hand, decrease the number of features by a negligible amount. On the other hand, it will significantly decrease the number of non-zero attribute values. Looking at the ranked list of words from another perspective, a significant majority of words appear only a few times. These words are thus often irrelevant for many tasks (e.g., classification, clustering, or association mining) and can be ignored. They also often represent some spelling errors so they actually work as a noise. By removing these words from further processing, a substantial reduction of the number of features can be achieved.

The large number of features and the implications of Zipf's law also lead to an additional problem. When the number of words in the documents is considered, it quite small compared to the number of all unique words in a document collection. Looking at the previously mentioned collection of customer reviews, we were able to identify almost 1,000 unique words for the small set consisting of 100 documents. Taking the average length of 25 words, about 2.5% of the vector elements have a non-zero value (when all words in a document are distinct). In the case of the largest collection of 100,000 documents, the percentage of non-zero items is much smaller, about 0.05%. We therefore say that the vectors are very sparse which is a problem

for some tasks, computing similarity in classification or clustering [177], for example. The sparsity can also be seen in the columns of the document-term matrix because a significant number of words appear only a few times. The sparsity generally means that it is, at times, quite complicated to identify some patterns in the data [87].

Since word order is not considered, the bag-of-words model has several disadvantages. For example, two sentences with the same words that appear in a different order will be treated as identical. When word order in a comparative sentiment analysis task is not taken into consideration, the sentences A is better than B and B is better than A will look the same although they should lead to opposite conclusions.

The representations of individual words also bear little semantics. Each word in the bag-of-words model is just one column in the document-term matrix where the order of columns is not important. When columns A and B are closer to each
other than columns A and C , it does not mean that the words A and B are more similar in their meaning or grammatical form, for example, than the words A and C . In the vector space model, all words create one dimension that is equally important as the others. The distance from the beginning of the vector space of a position representing, for example, one occurrence of the word A is the same as the distances of other positions representing one occurrence of any other word.

The issue of the large number of features can be solved by feature selection methods (see Chapter 14). The bag-of-words model can be also used to derive new representations (using, for instance, latent semantic analysis or neural models creating word embeddings) with fewer dimensions, thus eliminating the above mentioned problems. These methods are briefly described in Chapter 13. The loss of context can be eliminated by working with sequences of words known as n-grams (we therefore sometimes talk about bag-of-n-grams) or by introducing new attributes bearing information about the context of a word (for example, what is the preceding word, what is the following word, what is the part-of-speech of the previous word, etc.). Here, however, problems of high dimensionality and sparseness are often intensified.

Despite the numerous problems and limitations of the bag-of-words model, it brings surprisingly good results in many tasks. In situations, especially when there are fewer features, where there is no correlation between them or when we do not want to take it into account, the bag-of-words approach is appropriate. Sparse vectors can be also used to learn dense embedding through training a neural network [101]. For example, combining the embedding vectors of

words (through averaging them, for instance) from a document into one vector enables application of classical machine learning algorithms for classification.

3.4 Document Features

The form taken by the features characterizing documents depends on the problem to be solved. For example, using characters as features might be somehow useful for a spam filtering task where the presence of some special characters or too many identical characters can indicate a spam e-mail. On the other hand, in an information extraction problem where some kind of semantic information understandable by people is expected as the output, characters as features do not make much sense. Here, words or concepts play a more important role.

In most cases, the following feature types can be distinguished [87]:

- *Characters, character bi-grams, tri-grams, n-grams*: In spite of being the most complete representation of textual data, characters are not often used as features. The reason is quite simple - one character does not bear much semantic information, so any interpretation is quite complicated, especially when the positional information, i.e., information about the context of a character, is lost (we can thus talk about a bag-of-characters representation).
- *Words*: Words are the most natural and straightforward features that can be derived from texts. A word is a sequence of symbols, typically letters, that is used in the sentences of a given language. It has a semantic, grammatical, and phonological unity and is usually separated from other words by spaces in writing [74]. This is how are words understood by most western languages. On the other hand, texts in, for instance, Chinese or Arabic do not usually contain spaces between words so it is not easy to visually identify their borders.
  A *dictionary* can be used to find out what is and what is not a word in a given text. It is quite typical that a dictionary, a list of words that are available in the given language, exists for a natural language. The dictionaries might contain only the basic form of the given word, called lemma. For example, the lemma for does, doing, done, or did is do. Therefore, the absence of a particular sequence of characters in a dictionary does not necessarily mean that it is not a proper word.

  What about other parts of texts, like numbers, temporal expressions, currency symbols, and punctuation (comma, period, question mark,

quotation marks, etc.)? It generally depends on the task to be performed. Sometimes, a particular number or date can be important. If the number is, for example, the price of a product, we can use a specific value to find out whether the product is expensive or cheap. Sometimes it is enough to know that the text shows specific price.

One of the disadvantages of using words is that, without a context, the possible meaning of a word may not be obvious.

- *Terms*: Terms are single words or multiword expressions formed of words from the text. When looking at the fictive news headlines mentioned earlier in this chapter, terms like Las Vegas, ice hockey, FIFA World Cup, or the National Lottery can be identified. Since more words are treated as one unit, the ambiguity of words is reduced.

  Term extraction is related to the Named Entity Recognition (NER) problem, which is a subtask of Information Extraction. Its goal is to identify information units like names (typically persons, organizations, and location) and numeric expressions including time, date, money, and percent expressions [205].

- *Concepts*: Concepts are features that are generated from the text based on the occurrence of words, phrases, or more complicated structures. The concepts do not have to appear directly in the text. In the case of news headlines, for example, concepts like sport or gambling can be identified.

  Concepts, together with terms, have the most expressive semantic value. Concepts enable better handling of synonymy, polysemy, hyponyms, or hypernyms. On the other hand, they require more effort to be extracted. They are also often domain dependent.

  Assigning correct concepts often involves cross-referencing against a source of external knowledge which can be the knowledge of a domain expert, an existing ontology or lexicon, or an annotated collection of training documents used to train an algorithm assigning concepts.

Characters or words can be combined to form character or word n -grams. A character n -gram is a sequence of n consecutive characters. A few character 3-grams created from this sentence include "A f", " fe", "few", "ew ", "w 3",

etc. Word 2-grams from this sentence include "Word 2-grams", "2-grams from", "from this", etc.

The character n -grams are able to capture whole words ("few") as well as word class (e.g., "ed ", "ing"). A great advantage of character n-grams when compared to word n-grams is that they do not create as many dimensions (the sparsity of data is lower) because there exist fewer character combinations than word combinations [139]. Character n-grams have been successfully used for many tasks, including language identification [38], authorship identification [118], malicious code detection [223], and plagiarism detection [17].

Word n-grams can help in capturing multiword phrases [163] and have been successfully used in sentiment analysis [273].

An interesting collection of word n-grams, together with their frequency of usage in books, is provided by Google. Their new edition of the Google Books Ngram Corpus provides syntactically annotated n-grams for eight languages and covers about 8% of books that have been ever published (more than 8 million books) [172].

N -grams can be also understood as a simple statistical language model that models probabilities of sequences of characters or words. In other words, given a sequence of previous words, the model can help us predict a word. As such, it can be successfully used in task, such as machine translation, spelling correction, or speech recognition [138].

3.5 Standardization

Text data is commonly presented in some kind of semi-structured representation. A web page containing the most recent news might include several news features consisting of a heading, list of keywords, a few paragraphs of text, images with captions, etc., all enclosed within or similar tags with defined classes.

For example, an XML file containing news from Reuters Corpus Volume 1 (RCV1) [167] consists of elements containing the publication date, related topic, mentioned people, places or organizations and the text consisting of a title, date line, and body (see Figure 3.4).

A JSON (JavaScript Object Notation) file stores the data in a human readable format as objects consisting of name/value pairs where the value can be a scalar (string, number, logical value), another object, or a list of values [79].

With a semi-structured document, only the content of some selected ele-

```
<?xml version="1.0" encoding="iso-8859-1" ?>
<newsitem itemid="2286" id="root" date="1996-08-20" xml:lang="en">
  <title>MEXICO: Recovery excitement brings Mexican markets
         to life.
  </title>
  <headline>Recovery excitement brings Mexican markets
           to life.
  </headline>
  <byline>Henry Tricks</byline>
  <dateline>MEXICO CITY</dateline>
  <text>
    <p>Emerging evidence that Mexico's economy was back on the
       recovery track sent Mexican markets into a buzz of excitement
       Tuesday, with stocks closing at record highs and interest
       rates at 19-month lows.
    </p>
    <p>&quot;Mexico has been trying to stage
    ...
    </p>
  </text>
  <copyright>(c) Reuters Limited 1996</copyright>
  <metadata>
    <codes class="bip:countries:1.0">
      <code code="MEX">
        <editdetail attribution="Reuters BIP Coding Group"
                    action="confirmed" date="1996-08-20"/>
      </code>
    </codes>
    <codes class="bip:topics:1.0">
      <code code="E11">
        <editdetail attribution="Reuters BIP Coding Group"
                    action="confirmed" date="1996-08-20"/>
      </code>
      ...
    </codes>
    <dc element="dc.publisher" value="Reuters Holdings Plc"/>
    <dc element="dc.date.published" value="1996-08-20"/>
    ...
  </metadata>
</newsitem>
```

Figure 3.4: An example of an XML file containing a newspaper article from the RCV1 document collection.

ments might be important for a specific task. For example, to train a classifier

to categorize newspaper articles, only the content of elements <code> in the <codes class="bip:topics:1.0"> section, <title>, and <text> is relevant. The content of <title> and <text> will be the base for deriving document features, the content of <code> element will be used to specify the class labels (one or more) for the documents.

The XML library can be used to process XML (and HTML) documents. The xmlToList() function converts an XML document of node to a list that directly contains the data. The list elements correspond to the children of the top-level node and the .attrs element is a character vector of attributes from the node.

```
> library("XML")
> data<-xmlToList("Reuters/CD1/2286newsML.xml")
> names(data)
[1] "title" "headline" "byline" "dateline" "text"
[6] "copyright" "metadata" ".attrs"
> data["title"]
$title
[1] "MEXICO: Recovery excitement brings Mexican
    markets to life."
> data$metadata$codes$code$.attrs
    code
"MEX"
```

The following example (from [143]) shows how to extract information about reviews of conference papers. The file with the information in the JSON format contains one object with key papers. The value associated to this key is an array where every element represents one paper. Every paper has a numeric identifier, a string value representing a preliminary decision on the paper, and the list of reviews. The reviews contain information about reviewers' confidence, evaluation of the paper on a given numeric scale, the number of the review, language, text of the review, additional remarks to the conference committee, date of the reviews, and authors' evaluation of the review. An excerpt from the JSON file can be found in Figure 3.5.

The fromJSON() function from the rjson package converts a string containing a JSON object, the content of a file, or the content of a document with a given URL to an R object. The type of the object is list. Its elements are numbered when they come from a list of values in a JSON object and have

names when they are converted from an object with name/value pairs.

```
{
 "paper": [
  ...
  {
   "id": 8,
   "preliminary_decision": "accept",
   "review": [
      {
       "confidence": "4",
       "evaluation": "1",
       "id": 1,
       "lan": "en",
       "orientation": "0",
       "remarks": "",
       "text": "The paper describes an experience concerning...",
       "timespan": "2010-07-05"
      },
      {
       "confidence": "4",
        "evaluation": "2",
        "id": 2,
        "lan": "en",
        "orientation": "1",
        "remarks": "",
        "text": "This manuscript addresses an interesting solution...",
        "timespan": "2010-07-05"
      },
  },
  {
   "id": 12,
   "preliminary_decision": "accept",
   "review": [
      {
       "confidence": "4",
       "evaluation": "2",
       "id": 1,
       "lan": "es",
       "orientation": "2",
       "remarks": "Se trata de un articulo que presenta un tema ...",
       "text": "Es un articulo que presenta un tema novedoso ...",
       "timespan": "2010-07-05"
      },
      ...
  },
  ...
 ]
}
```

Figure 3.5: An example of a JSON file containing information about reviews of papers from a conference.

```
> library("rjson")
> data <- fromJSON(file="reviews.json")
> names(data)


[1] "paper"
> names(data[['paper']][[1]])
[1] "id" "preliminary_decision"
[3] "review"


> names(data[['paper']][[1]][["review" ]][[1]])
[1] "confidence" "evaluation" "id" "lan"
[5] "orientation" "remarks" "text" "timespan"



> # a piece of text of the first review of the first paper
> substr(data[["paper"]][[1]][["review"]][[1]][["text"]],
+ 1, 60)
[1] "- El articulo aborda un problema contingente y muy
relevante"



> # the language of the first review of the first paper
> data[["paper"]][[1]][["review"]][[1]][["lan"]]
[1] "es"



> # a decision on the first paper
> # same as data[[1]][[1]][[2]]
> data[["paper"]][[1]][["preliminary_decision"]]
[1] "accept"
```

The following commands convert the data to a data frame with two columns, Text and Evaluation. The data can be used, for example, in a

classification task where the evaluation will be predicted according to the text of reviews.

```
# creating an empty data frame
d <- data.frame("Text"=c(),
        "Evaluation"=c(),
        stringsAsFactors=FALSE)
for (i in 1:length(data[["paper"]])) {
    # looking at the number of reviews of the ith paper



    nr <- length(data[["paper"]][[i]][["review"]])
    if (nr==0) {next}
    for (r in 1:nr) {
        # adding the text and evaluation to the data frame d
        d <- rbind(d,
            data.frame(
                "Text"=data[["paper"]][[i]][["review"]]
                    [[r]][["text"]],
                "Evaluation"=data[["paper"]][[i]]
                        [["review"]][[r]]
                        [["evaluation"]]
                )
            )
    }
}
```

3.6 Texts in Different Encodings

Texts are stored in computer memory as a sequence of bits. Each character then usually occupies a certain number of bytes. To order to be stored, a character needs to be encoded to a number. It needs to be decoded to display a character.

In the past, seven bits enabling the storage of 128 distinct characters were needed to encode a character. This was enough for English letters, digits, and some other common symbols like punctuation.

However, for some other languages, like German, Spanish, French, Czech, etc., characters with accents were needed. It was not possible to encode all these characters to a 7-bit character, so eight bits were used. Different languages had different encodings. For some languages, like Chinese, even eight bits were not enough [210]. This problem can be solved using the Unicode encoding, which is a de facto standard in simultaeously encoding many languages and scripts [109].

Knowledge of text encoding is necessary for some of the preprocessing steps, like tokenization, for example. In some languages, characters encoded by some numbers are regular word characters while in others they are punctuation characters [210].

Document encoding can be explicitly indicated as, for example, in some metadata or headers of a document. In certain contexts, an explicit or implicit agreement on a specific encoding might be made as well. When no such information is available, a user or computer must make a guess based on an inspection of the text [149].

To guess a document encoding in R , the stri_enc_detect() function from package indexR packages!stringistringi can be used. The function accepts a character vector and returns a list of guessed encodings, language, and confidence level for each string. It currently supports about thirty different character sets and languages.

```
texts <- c("What is your name?",
        "¿Cómo te llamas?",
        "Comment tu t'apelles?")
stri_enc_detect(texts)
[[1]]
    Encoding Language Confidence
1 ISO-8859-1       en       0.47
2 ISO-8859-2       hu       0.31
3      UTF-8                0.15
...

[[2]]
    Encoding Language Confidence
1      UTF-8                0.80
2 ISO-8859-1       es       0.47
3 ISO-8859-2       hu       0.15
```

```
...
[[3]]
    Encoding Language Confidence
1 ISO-8859-1       fr       0.71
2      UTF-8                0.15
3 ISO-8859-2       hu       0.14
```

Texts can be converted from one encoding to another using the standard iconv() function. The list of available encodings is provided by the iconvlist() function.

```
> x<- "Gr\xfc\xdf Gott"
> x
[1] "Grüß Gott"
> Encoding(x)
[1] "unknown"
> Encoding(x) <- "latin1"
> Encoding(x)
[1] "latin1"
> xx <- iconv(x, from="latin1", to="UTF-8")
> Encoding(xx)
[1] "UTF-8"
```

3.7 Language Identification

Some of the preprocessing steps, like tokenization, stemming, or part of speech tagging are language dependent. Language identification is actually a categorization problem - a language label is assigned to a text.

A straightforward approach can compare words of a text with dictionaries of known languages and assign the language with the highest match as the correct one. Typically, texts are compared with only the most frequent words in a language. The accuracy of these methods is, however, not very high, especially for short texts [106].

Other approaches work with variable or fixed length character n-grams. The n-grams in a text are compared to an n-grams based model of a language. The model can be based on simple frequencies of n-gram occurrences [44] or can have the form of a probabilistic Markov model [215].

A comparison of some popular language identification algorithms can be found in [131].

To determine a language, for example, R provides, the cld2 (Google's Compact Language Detector 2) package and its function detect_language().

```
texts <- c("What is your name?",
    "¿Cómo te llamas?",
    "Comment tu t'apelles?",
    "Wie heißt du?",
    "Jaké je tvoje jméno?")
detect_language(texts)
[1] "en" "es" "fr" "de" "cs"
```

3.8 Tokenization

Tokenization is the process of splitting a document into pieces of texts known as tokens. These tokens are often the words contained in the text. In most European languages where the words are space delimited, the task seems to be quite simple - to split the text at the places where are white spaces. In some other languages, like Chinese, where there are no spaces between words, the text needs to be analyzed in greater depth.

Even in European languages, the process is usually not that straightforward. Punctuation (including brackets, apostrophes, dashes, quotation marks, period, exclamation mark) is usually understood as something separating the tokens. Besides removing white spaces, punctuation marks are usually removed too which is probably the simplest tokenization approach [275,211].

There are, however, many exceptions and special problems related to different languages. Even for English, tokenization is not an easy task. The following text mentions a few typical problems.

A period is usually understood as the end of a sentence. It can be, however, used for other purposes, like in abbreviations (Mr., etc., U.S.A.) or in numbers in English (1.23). In such cases, the period is often a part of the tokens.

Apostrophes can be used in contractions (I'm, haven't), genitive forms of nouns (Peter's), or when omitting first two digits in a year ('90s). We then need to decide whether the apostrophe should be removed (haven't $\rightarrow$ haven $+t$), retained, or replaced ('90s $\rightarrow$ 1990s). In many situations, it is still not

clear what should happen. For example, should doesn't be transformed to doesnt, doesn $+t$, or does $+n$ 't?

Several words can be also connected by hyphens to create compound words (state-of-the-art, bag-of-words, two-year-old, sugar-free, long-lasting). Should we treat each part of the compound expression separately or the whole expression as one token?

Special attention should be given to numbers. Numbers can be written in many different ways, including a decimal point (comma in some languages), + or - sign, semilogarithmic notation, etc. When a phone number is treated as one token we cannot see, for example, an area code (00420-777777777 vs. $00420 + 777777777$). The same might be applied to dates, 2018-02-14 can be treated as one token as well as $2018 + 02 + 14$ so we have information about a year, month, and day.

Other pieces of text that require special attention are, for example, e-mail addresses, URLs, special names ($C++$, Bell&Ross) among others.

Some languages, like German, often use compound words that are not hyphenated. For example, the longest German word in 2013 [265] Rindfleischetikettierungsüberwachungsaufgabenübertragungsgesetz refering to the law for the delegation of monitoring beef labelling consists of ten words. Some of them are prefixes, but can also be prepositions and are counted as words here. Thus, the long word consists of ten words.

It is obvious that tokenization is a language specific task. The language of a text must therefore be known for tokenization that is not naive. The rules defining some specific tokens in a language must be clearly stated. The process of tokenization is also influenced by the used text encoding because delimiters and special symbols occupy different positions in different character tables [211].

3.9 Sentence Detection

In some tasks, splitting a document into smaller units, like sentences, can be beneficial. It seems that finding symbols which normally end sentences, like ., ?, or ! should be enough. These symbols can, however, also be parts of other expressions (for example, a period can be a part of an abbreviation or a number). A terminating symbol can be also omitted, like in the case when a sentence ends with an abbreviation.

A human expert can define a set of rules to be used to separate sentences (e.g., the symbol, like a period, must be followed by a whitespace and an

uppercase letter). As manual creation of such rules need not be the best approach, an algorithm can be trained to detect sentences from labeled data [225].

In R, sentences can be extracted from a text using the udpipe() function from package udpipe. For more details, see the example at the end of this chapter.

```
> unique(udpipe("I'm Mr. Brown. I live in the U.S.A.",
+               object="english"
+               ) ['sentence']
+       )
                sentence
1       I'm Mr. Brown.
6 I live in the U.S.A.
```

3.10 Filtering Stop Words, Common, and Rare Terms

Some of the words that are too common do not usually contribute (or contribute only negligibly) to achieving a certain goal. For example, in the query Find *a* hotel in Prague submitted to a web search engine, the word *a* does not influence what will be retrieved by the search engine. The retrieved results will be the same no matter whether *a* is or is not in the query. The reason is simple - almost all documents written in English contain at least one occurrence of the word *a*. On the other hand, the word Prague is very important because without it the search engine would return information about hotels in many other locations.

Similar importance of words is apparent in other text mining tasks, such as classification or clustering. When categorizing, for example, newspaper articles to categories, the occurrence of word the will not play an important role because articles from all categories are very likely to contain this word. In a clustering process, when calculating the similarity between two documents, the occurrence of the word the will contribute to the similarity measure with the same value for almost all document pairs.

It is obvious that the words not contributing to achieving a certain goal are useless and do not have to be considered in further processing.

The words that are not important for a certain task are known as stop words. Very often, they are the most frequent words in a given language. Typical stop lists for English contain words like the, of, and, by, with (grammatical articles, prepositions, conjunctions, auxiliary verbs, pronouns, articles). Many stop word lists can be found on the web or incorporated in various tools and in the libraries of programming languages. Table 3.1 shows how the numbers of stop words for different languages can differ (the table is based on stop lists in python's NLTK library v. 3.4). It is, however, obvious that the number of stop words is usually around a few hundred. It should be noted that the available stop word lists sometimes suffer from surprising omissions and inclusions, or are incompatible with particular tokenizers [207]. Various stop word lists in R can be obtained from the function stopwords() from the tm package.

The stop word list length, when compared to the size of the dictionary, which can be a few hundreds of thousands, is negligible. The number of features thus does not decrease significantly when stop words are removed. A massive reduction is, however, noticeable in the total number of words (which is related, for example, to the memory requirements). According to Zipf's law, ten most frequent words represent approximately one third of all words in a corpus.

A stop list can also be easily created by simply counting the frequencies of words in a corpus and looking at the most frequent words. It is, however, better to carefully examine the frequent words so that important words are not lost.

What is and what is not a stop word might depend on a domain. In a collection of hotel accommodation reviews, the word hotel would probably be very frequent and will not be useful in, for example, determining whether a review is negative, positive, or neutral. In the following example, reviews from booking.com (for a detailed description of the data, see [295]) are processed. The number of all words, unique words, and global words frequencies are calculated from a matrix representing the documents. The frequencies together, with relative frequencies of the ten most frequent words are printed. Subsequently, the 50 most frequent words are displayed without their frequencies. We can see that the list contains typical English stop words as well words typical for the hotel accommodation domain. From a sorted list of words together with their frequencies, a graph demonstrating Zipf's law can be easily generated (with logarithmic axes).

Table 3.1: Numbers of stop words for different languages in stop lists in Python's NLTK library v. 3.4.

Language	Number of Stop Words
English	179
German	231
Italian	279
French	155
Spanish	313
Russian	151

```
> library(tm)

> text <- readLines("reviews-booking.txt")
> corpus <- VCorpus(VectorSource(text))
> corpus <- tm_map(corpus, removePunctuation)
> corpus <- tm_map(corpus, stripWhitespace)
> corpus <- tm_map(corpus, content_transformer(tolower))
> dtm <- DocumentTermMatrix(corpus)

> mat <- as.matrix(dtm)

> word_frequency <- colSums(mat)
> number_of_words <- sum(word_frequency)
> unique_words <- length(word_frequency)
> paste("The number of all words: ", number_of_words)
[1] "The number of all words: 394993"
> paste("The number of unique words: ", unique_words)
[1] "The number of unique words: 19528"

# printing a word together with its rank, total frequency,
# and relative frequency
> words_sorted <- sort(word_frequency, decreasing = TRUE)
> for (i in 1 : 10) {
+ print(paste("word: ", i, ": ",
+             names(words_sorted[i]),
+             ", freq.: ", words_sorted[i],
+             ", rel. freq.: ",
```

```
+              round((words_sorted[i]/number_of_words),3),
+              sep=""
+             )
+     )
+ }
[1] "word 1: the, freq.: 33313, rel. freq.: 0.084"
[1] "word 2: and, freq.: 18681, rel. freq.: 0.047"
[1] "word 3: was, freq.: 13445, rel. freq.: 0.034"
[1] "word 4: very, freq.: 8644, rel. freq.: 0.022"
[1] "word 5: room, freq.: 6969, rel. freq.: 0.018"
[1] "word 6: hotel, freq.: 6742, rel. freq.: 0.017"
[1] "word 7: for, freq.: 6092, rel. freq.: 0.015"
[1] "word 8: staff, freq.: 5447, rel. freq.: 0.014"
[1] "word 9: good, freq.: 4819, rel. freq.: 0.012"
[1] "word 10: not, freq.: 4361, rel. freq.: 0.011"
# printing 50 most frequent words
> names(words_sorted[1:50])
 [1] "the"         "and"       "was"   "very"
 [5] "room"        "hotel"     "for"   "staff"
 [9] "good"        "not"       "were"  "location"
[13] "breakfast"   "with"      "but"   "clean"
[17] "friendly"    "from"      "nice"  "had"
[21] "there"       "helpful"   "rooms" "that"
[25] "have"        "excellent" "you"   "this"
[29] "great"       "our"       "are"   "all"
[33] "comfortable" "stay"      "would" "they"
[37] "small"       "close"     "only"  "one"
[41] "well"        "could"     "which" "food"
[45] "service"     "city"      "also"  "night"
[49] "bathroom"    "when"
# plotting the curve demonstrating the Zipf's law
> plot(words_sorted, type="l", log="xy",
+ xlab="Rank", ylab="Frequency")
```

Special attention should be paid to stop words in multiword expressions and to words that are somehow affected by the occurrence of a stop word. Consider the following phrases: room with balcony and room without balcony, or good and not good. Leaving the stop words (with, without, not) out

of consideration, the meaning of the phrase will change significantly (in fact, it will be the opposite). There are also phrases consisting solely of stop words, like The One (a movie), to be or not to be (a famous phrase from Shakespeare's Hamlet), or The Who (an English rock band). Weiss et al. [275] thus recommend the removal of stop words after the features for a document set are created and other preprocessing techniques (e.g., stemming) have been applied.

While stop words appear at the top of a list of words ordered by their frequency, the words at the other end of the list can also be eliminated. Consider a classification problem where there are 1,000 documents in two classes, C_1 and C_2. Only one document, d from the entire collection contains one specific word w. The document belongs to class A. The word w is definitely helpful and can be used when assigning documents to A. The rule if a document contains w it should be assigned to A will cover 1/1000 of the training examples and is therefore not too general. Having too many such rules will very likely lead to overfitting of a classifier which is not desirable. Not considering w by a classifier can cause $1/1000 = 0.1\%$ error on the training data which is also negligible.

In the calculation of the similarity of two documents during clustering, the words with low frequencies will not have an effect on the results because there is actually no similarity in the given dimension when one document contains a word and the other does not.

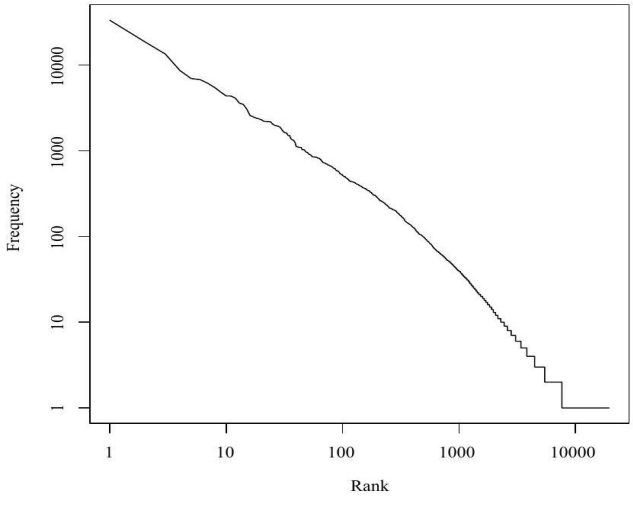


Figure 3.6: Graphical demonstration of Zipf's law.

It seems that removing rare words is a good idea for most of the tasks. Sometimes, the computational complexity of a problem can be lowered significantly, especially for algorithms, where the complexity grows exponentially with the number of features. On the other hand, in an information retrieval task, not considering those rare words (for example, a specific name of a person, place, or company) can result in the failure to achieve a certain goal. Here, the document containing one specific word will not be found by a search engine. Indexing rare terms thus increases the success rate of a search engine but, on the other hand, might decrease its performance because more terms would need to be stored and processed.

3.11 Removing Diacritics

Diacritics (diacritical marks, accents) are symbols added to letters, usually above or below. Diacritics often changes the way a sound is pronounced (e.g., longer, with an accent). In some languages, like in Vietnamese, Czech, Spanish, or French, diacritics are quite common. In some situations, they help distinguish between two completely different words like in Spanish - cómo (how) and como (I eat), or French - cote (quota), coté (respected), côte (coast), or côté (side). The removal of accents can thus, sometimes lead to the loss of very important information.

It is, therefore, necessary to look at the data and think about the process of its creation. In the past, many SMSs and e-mails were written without diacritics, so some normalization could be useful.

In English, almost no accents are used and their potential removal does not usually cause major problems (e.g., naïve → naive).

In R , most versions of the iconv() function enable diacritics removal by appending //TRANSLIT to the output encoding.

```
> x<- "Dobrý den, člověče!"
> iconv(x, from="CP1250", to="ASCII//TRANSLIT")
[1] "Dobry den, clovece!"
```

3.12 Normalization

The goal of normalization is to convert tokens that look differently (are, for example, written in a different letter case) but have the same meaning to the

same form [263].

3.12.1 Case Folding

Case folding is the process of converting all characters of a word to the lower or upper case (the lower case is more common [62]). When two or more words with the same meaning are written with different casing, the number of unique words increases unnecessarily and it is, therefore, favourable to convert them to one form. The reasons why the same word is written with different casing might include the position at the beginning of a sentence, presence in a title, the intention to emphasize a word, etc.

Some words, on the other hand, require capitalization and folding the case in these situations changes the meaning of the word. Typically, names of people, places, companies, etc. are written with the first letter capitalized. For example, bush in the middle of a sentence means a small, woody plant while Bush would probably mean the former American president (when Bush is at the beginning of a sentence, it could be either). After conversion to lower case and not being aware of the context, one cannot distinguish whether a text is about plants or politics according to the occurrence of word bush. Abbreviations, although sometimes looking like regular words, are written in the upper case. SMART (System for the Mechanical Analysis and Retrieval of Text) when converted to lower case is a synonym for intelligent.

Case folding can be problematic in some specific languages. For example, in French, accents are optional for uppercase. The word PECHE can be converted to pêche which means fishing, peach or péché meaning sin.

Capitalization plays an important role in some languages [102]. For example, all nouns are capitalized in German. For instance, Recht as a noun means law while recht as an adjective means right. The letter β also exists only in lower case.

Intelligent case folding is a machine learning problem. As an alternative, a simple approach can convert all words at the beginning of sentences as well as the words from titles and headings to lower case. The case of other words can be retained [6].

3.12.2 Stemming and Lemmatization

A word is the smallest lexical unit of a language that can be used in isolation. A morpheme is the smallest unit of a word that carries some semantic or grammatical meaning. Morphemes typically include prefixes, suffixes, and a root. For example, the word unexpected consists of three morphemes - a prefix

un, a root expect, and a suffix ed. Some morphemes can be used in a language by themselves (expect) while not the others (un, ed). The former are known as free forms, the latter as bound forms [11].

Stems are parts of words that carry the basic meaning. When a stem consists of a single morpheme, it is identical to the root. Free stems can occur alone whereas bound stems cannot.

Words are formed in a word-formation process using several rules. For example, a plural morpheme *s* can be added to a singular noun to create a plural noun (e.g., book → books). One major morphological operation is the use of inflection which is the addition of a plural, possessive, plural+possessive, comparative, superlative, 3rd person singular, past tense, past participle, or present participle forms to a stem. The second type of word-formation operations is derivation which includes adding adjective derivational affixes (day → daily, depend → dependent, help → helpful etc.), adverb derivational affixes (slow → slowly, or clock → clockwise and many others [126]. Inflection never changes the category of a word (e.g., a noun will be still a noun after adding a suffix) while derivation can change the category (a noun can become an adjective after adding a suffix).

It is obvious that the semantic meaning of some words, when created from the same stem, is very close. When a newspaper article contains one of the words sport, sports, sporting, sported, or sporty, it is quite likely to be an article from the sports category. However, for a computer, all five words are different and to assign an article correctly to the category, all five words need to be connected to it, for example, in a classification rule.

The goal of the procedure known as stemming is to convert all different forms of a word to one common form known as the stem.

To find a correct stem, a morphological analysis of the word is often necessary. However, many stemmers (a stemmer is a function or a program performing stemming) employ simple rules to strip affixes. This can lead to a situation that the difference between, for example, a noun and verb disappears - a noun driver and verb driving will be stemmed to driv. We can also see that driv is not a correct dictionary word. Some of the stemmers are not constrained to produce only real words [189]. This is usually not considered a deficiency because the word is often used as a feature in a machine learning task and not as a piece of a message for a human. At the same time, when a stem changes with the addition of a suffix (e.g., index → indices), a simple stemmer will not convert different variants to a common one.

It is obvious that a stemming process is not error-free. The errors include understemming (the found stem does not cover all word variants) and

overstemming (a word is shortened too much so it covers words with different meanings). Understemming is usually achieved by light stemmers that prefer the precision over the recall while aggressive stemmers preferring the recall often overstem the words [39].

The most famous stemming algorithm for English text, belonging to the group of rule-based stemmers, is Porter's algorithm [217]. It uses five groups of rules that are successively applied to strings, for example, SSES → SS (caresses → caress), ATIONAL → ATE (relational → relate), ALIZE → AL (formalize → formal, or *AL* → "" (revival → reviv). The rules can contain conditions, like whether the stem contains a vowel, ends with a double consonant, is longer than a given length etc.

Porter [218] also developed a system for defining stemming algorithms. The rules of stemming algorithms are expressed in a natural way in a simple language known as Snowball. A Snowball compiler then translates a Snowball definition into an equivalent program in ANSI C or Java.

Dictionary stemmers need a dictionary containing all variants (inflected forms) of a word together with their stems. This method is not suitable for languages with many words and for heavily inflected languages [140].

Rule based and dictionary stemmers require that the rules be constructed by linguistic experts for a specific language. As they are not able to handle situations which are not covered by the rules or the dictionary, statistical or machine learning approaches would be better in some situations. For example, the algorithm of Bacchcin, Ferro, and Melucci et al. [15] estimates the most probable split into the stem and suffix from all possible splits for each word in the full corpus. In [39], the authors clustered together words occurring in similar contexts and sharing a common prefix of at least a minimum length. Such words are expected to be semantically and lexically similar; this can be enough to replace them by one form. They are, however, taken as the training data for a classifier which learns stemming rules so that processing unseen data is possible.

Lemmatization converts a word to its dictionary form known as the lemma. To be able to perform such a conversion, deeper knowledge than that required for stemming is needed. A lemmatizer usually needs a complete vocabulary and morphological analysis to correctly lemmatize words [185]. The words like better or thoughts can be converted to their correct forms good and think, which would not be achieved by a stemmer. Attempts to train a lemmatizer, like in [14, 136], have also been made but they are not error free.

3.12.3 Spelling Correction

Mistypings are common problems in any written text. The problem is that they bring noise to the data which can complicate the achievement of satisfactory results in some tasks. Spelling correction is a solution to this problem. Generally, the process of spelling correction involves detection of an error, generation of candidate corrections, and ranking of candidate corrections [155].

Mistypings can be detected by comparing each word to an existing dictionary or looking at every character n-gram (n is usually 1,2 , or 3) to determine whether such an n-gram exists for a given language.

When an error is found, our response is to correct it. The correction can be done with or without taking into consideration, the context of the error. The methods which do not take the context into consideration suppose that most of the errors are caused by insertion, deletion, substitution, and transposition; they appear in the middle of a word, can be related to a computer keyboard arrangement, etc.

Sometimes, knowledge of the context in which the error appears can help in choosing the right candidate. The context can also help in finding errors when a regular but incorrect word is used instead of a correct word. For example, in the sentence Three are two people form Prague here, only regular English words are used, but three and form should be replaced with there and from. The context specific spelling correction algorithms can rely on word association (which candidate correction has a better semantic fit with the words around the misspelling), word repetition (words occurring multiple times in a text can help in finding appropriate corrections), and topical bias (certain correction candidates could be preferred by considering words that are especially relevant to the topic of the text) [90].

Package hunspell provides a functionality that can be used when identifying mistyping and correcting the mistakes. Function hunspell_parse() tokenizes a text so it can later be checked for errors. As a parameter, the format of input text can be passed (text, latex, man, html, or xml formats are supported).

```
> t <- c("How are you these days?",
+        "I'm fine, thnx!",
+        "It is <emph>grrreat</emph>, right?")
> words <- hunspell_parse(t, format="html")
> words
[[1]]
```

```
[1] "How"   "are"   "you"   "these"   "days"
[[2]]
[1] "I"   "m"   "fine"    "thnx"
[[3]]
[1] "It"      "is"    "grrreat" "right"
```

The hunspell_analyze() function gives information about stems and suffixes of words. To get only words stems, we can use function hunspell_stem().

```
> print(hunspell_analyze(words[[1]]))
[[1]]
[1] " st:how"
[[2]]
[1] " st:are"
[[3]]
[1] " st:you"
[[4]]
[1] " st:these"
[[5]]
[1] " st:day fl:S"
```

To check whether a text contains misspelling, the hunspell_check() function can be called. It returns a logical value for each character string representing the presence of an error.

```
> correct <- hunspell_check(words[[3]])
> names(correct) <- words[[3]]
> correct
  It   is grrreat right
TRUE TRUE   FALSE  TRUE
```

A wrapper function hunspell() parses the text, finds incorrect words and returns a list of these words for each line. It effectively combines hunspell_parse() with hunspell_check() in a single step.

Function hunspell_suggest() provides a list of possible replacements for the supplied words (it is typically used to find alternatives for mistyped words).

```
> hunspell_suggest(words[[3]][!correct])
[[1]]
[1] "great"
[[2]]
[1] "right" "girth"
```

Spell checking using hunspell is dictionary based. The list of all available dictionaries is returned by the list_dictionaries() function.

```
> list_dictionaries()
[1] "en_AU" "en_CA" "en_GB" "en_US"
```

3.13 Annotation

The purpose of annotation is to distinguish tokens that look the same but have a different meaning. The context of a token is usually needed to reveal the correct meaning [263]. Annotation is, in fact, an opposite process to normalization.

3.13.1 Part of Speech Tagging

Part of speech classes contain words with the same grammatical function [241]. Open classes contain words that carry some important meaning and can accept new words. They typically include nouns, verbs or adjectives. They usually contain more words than the closed classes. The words from closed classes are relatively constant and new words are rarely added. They usually have a structural function, providing information on the syntactic relations existing among openclass words. Close classes usually contain prepositions, conjunctions, pronouns, or determiners [116]. Part of speech categories used in the famous Penn Treebank project can be found in Table 3.2. The Penn Treebank produced approximately 7 million words of part-of-speech tagged text between the years 1989 and 1996.

Table 3.2: An alphabetical list of part of speech tags from the Penn Treebank Project [259].

Tag	Description
CC	Coordinating conjunction
CD	Cardinal number
DT	Determiner
EX	Existential there
FW	Foreign word
IN	Preposition or subordinating conjunction
JJ	Adjective
JJR	Adjective, comparative
JJS	Adjective, superlative
LS	List item marker
MD	Modal
NN	Noun, singular or mass
NNS	Noun, plural
NNP	Proper noun, singular
NNPS	Proper noun, plural
PDT	Predeterminer
POS	Possessive ending
PRP	Personal pronoun
PRP$	Possessive pronoun
RB	Adverb
RBR	Adverb, comparative
RBS	Adverb, superlative
RP	Particle
SYM	Symbol
TO	to
UH	Interjection
VB	Verb, base form
VBD	Verb, past tense
VBG	Verb, gerund or present participle
VBN	Verb, past participle
VBP	Verb, non-3rd person singular present
VBZ	Verb, 3rd person singular present
WDT	Wh-determiner
WP	Wh-pronoun
WP$	Possessive wh-pronoun
WRB	Wh-adverb
.	Sentence-final punctuation
,	Comma
:	Colon, semi-colon
(	Left bracket character
)	Right bracket character
"	Straight double quote
'	Left open single quote
"	Left open double quote
,	Right close single quote
"	Right close double quote

Knowing a part of speech for the words contained in a document might help in extracting information-rich keywords from sentences [146], plagiarism detection [124], named entity recognition [168] and others. Part of speech tagging is also an important part of other tasks like syntactic parsing and machine translation [43].

Successful automatic part of speech taggers are usually built for resource-rich languages in a supervised way [288]. Rule-based taggers use a set of rules used to assign a correct part of speech tag. Statistical taggers try to assign a sequence of tags having the highest probability give by a language model to a text [37].

The package RDRPOSTagger performs part of speech tagging and morphological tagging based on the Ripple Down Rules-based Part-Of-Speech Tagger (RDRPOS). It contains pre-trained models for 45 languages.

The universal annotation (UniversalPOS) uses a reduced set of part of speech tags that is consistent across languages. Another approach (POS) uses an extended language specific tagset. The third option (MORPH) is very detailed morphological annotation. To find out which languages these options are available for, the rdr_available_models() can be used.

In order to annotate a text, a tagger needs to be created using the function rdr_model(). Its parameters are a language and the type of annotation. The function rdr_pos() then takes the created model of a tagger and text, performs part of speech tagging, and returns information about words and assigned tags.

```
> library(RDRPOSTagger)
> text <- "This book is about text mining and
           machine learning."
> tagger <- rdr_model(language="English",
+                     annotation="UniversalPOS")
> rdr_pos(tagger, x=text)
    doc_id token_id      token    pos
1       d1        1       This   PRON
2       d1        2       book   NOUN
3       d1        3         is    AUX
4       d1        4      about    ADP
5       d1        5       text   NOUN
6       d1        6     mining   NOUN
7       d1        7        and  CCONJ
8       d1        8    machine   NOUN
```

```
9       d1        9   learning   VERB
10      d1       10    . PUNCT
> tagger <- rdr_model(language="English",
+                     annotation="POS")
> rdr_pos(tagger, x=text)
    doc_id token_id token pos
1 d1 1 This DT
2 d1 2 book NN
3 d1 3 is VBZ
4 d1 4 about IN
5 d1 5 text NN
6 d1 6 mining NN
7 d1 7 and CC
8 d1 8 machine NN
9 d1 9 learning VBG
10 d1 10
> text <- "Este libro está escrito en inglés."
> tagger <- rdr_model(language="Spanish",
+ annotation="MORPH")
> rdr_pos(tagger, x=text)

\begin{tabular}{rrrrr}
& doc_id token_id & token & pos \\
1 & d1 & 1 & Este & DDOMS0 \\
2 & d1 & 2 & libro & NCMS000 \\
3 & d1 & 3 & está & VAIP3S0 \\
4 & d1 & 4 & escrito & VMP00SM \\
5 & d1 & 5 & en & SPS00 \\
6 & d1 & 6 & inglés & NCMS000 \\
7 & d1 & 7 &. & Fp
\end{tabular}
```